Operations Research

Publication details, including instructions for authors and subscription information:
http://pubsonline.informs.org

Strong Optimal Classification Trees

Sina Aghaei, Andrés Gómez, Phebe Vayanos

To cite this article:
Sina Aghaei, Andrés Gómez, Phebe Vayanos (2024) Strong Optimal Classification Trees. Operations Research

Published online in Articles in Advance 31 Jul 2024

. https://doi.org/10.1287/opre.2021.0034

https://pubsonline.informs.org/journal/opre

OPERATIONS RESEARCH
Articles in Advance, pp. 1–19
ISSN 0030-364X (print), ISSN 1526-5463 (online)

Methods

Strong Optimal Classification Trees

Sina Aghaei,[a,*] **Andrés Gómez,**[b] **Phebe Vayanos**[a]

[a] Center for Artificial Intelligence in Society, University of Southern California, Los Angeles, California 90089; [b] Department of Industrial and Systems Engineering, University of Southern California, Los Angeles, California 90089

*Corresponding author

Contact: saghaei@usc.edu, https://orcid.org/0000-0002-3394-8864 (SA); gomezand@usc.edu, https://orcid.org/0000-0003-3668-0653 (AG); phebe.vayanos@usc.edu, https://orcid.org/0000-0001-7800-7235 (PV)

Received: January 15, 2021
Revised: December 15, 2021; July 5, 2023; January 11, 2024
Accepted: January 11, 2024
Published Online in Articles in Advance: July 31, 2024

Area of Review: Optimization

https://doi.org/10.1287/opre.2021.0034

Abstract. Decision trees are among the most popular machine learning models and are used routinely in applications ranging from revenue management and medicine to bioinformatics. In this paper, we consider the problem of learning *optimal* binary classification trees with univariate splits. Literature on the topic has burgeoned in recent years, motivated both by the empirical suboptimality of heuristic approaches and the tremendous improvements in mixed-integer optimization (MIO) technology. Yet, existing MIO-based approaches from the literature do not leverage the power of MIO to its full extent: they rely on *weak* formulations, resulting in slow convergence and large optimality gaps. To fill this gap in the literature, we propose an intuitive flow-based MIO formulation for learning optimal binary classification trees. Our formulation can accommodate side constraints to enable the design of interpretable and fair decision trees. Moreover, we show that our formulation has a *stronger* linear optimization relaxation than existing methods in the case of binary data. We exploit the decomposable structure of our formulation and max-flow/min-cut duality to derive a Benders' decomposition method to speed-up computation. We propose a tailored procedure for solving each decomposed subproblem that provably generates *facets* of the feasible set of the MIO as constraints to add to the main problem. We conduct extensive computational experiments on standard benchmark data sets on which we show that our proposed approaches are 29 times faster than state-of-the-art MIO-based techniques and improve out-of-sample performance by up to 8%.

Funding: P. Vayanos and S. Aghaei gratefully acknowledge support from the Hilton C. Foundation, the Homeless Policy Research Institute, the Home for Good foundation under the "C.E.S. Triage Tool Research & Refinement" grant. P. Vayanos is funded in part by the National Science Foundation, under CAREER [Grant 2046230]. She is grateful for this support. A. Gómez is funded in part by the National Science Foundation under [Grants 1930582 and 2006762]. We would like to express our sincere thanks to the anonymous reviewers for their valuable and constructive feedback, which greatly improved the quality of this paper.

Supplemental Material: The online appendix and code files are available at https://doi.org/10.1287/opre.2021.0034.

Keywords: machine learning • optimal classification trees • mixed-integer optimization • Benders' decomposition

1. Introduction

1.1. Motivation and Related Work

Since their inception more than 30 years ago (Breiman et al. 1984), decision trees have become among the most popular techniques for interpretable machine learning (ML) (Rudin 2019). Typically, a decision tree takes the form of a binary tree. In each *branching* node of the tree, a binary test is performed on a specific feature. Two branches emanate from each branching node, with each branch representing the outcome of the test. If a datapoint passes (respectively, fails) the test, it is directed to the left (respectively, right) branch. A predicted label is assigned to all *leaf* nodes. Thus, each path from root to leaf represents a classification rule that assigns a unique label to all datapoints that reach that leaf. The goal in the design of optimal decision trees is to select the tests to perform at each branching node and the labels to assign to each leaf to maximize prediction accuracy (classification) or to minimize prediction error (regression). In practice, shallow trees are preferred as they are easier to understand and interpret. Thus, we focus on designing trees of bounded depth. Not only are decision trees popular in their own right, but they also form the backbone for more sophisticated machine learning models. For example, they are the building blocks for ensemble methods (such as random forests) that combine several decision trees and constitute some of the most popular and stable

machine learning techniques available (Breiman 1996, 2001; Liaw and Wiener 2002). They have also proved useful to provide explanations for the solutions to optimization problems (Bertsimas and Stellato 2021).

Decision trees are used routinely in applications ranging from revenue management to chemical engineering, medicine, and bioinformatics. For example, they are used for the management of substance-abusing psychiatric patients (Intrator et al. 1992), to manage Parkinson's disease (Olanow et al. 2001), to predict a mortality on liver transplant waitlists (Bertsimas et al. 2019b), to learn chemical concepts such as octane number and molecular substructures (Blurock 1995), and to evaluate housing systems for homeless youth (Chan et al. 2018). Moreover, tree ensembles can be used to reveal associations between micro RNAs and human diseases (Chen et al. 2019) and to predict outcomes in antibody incompatible kidney transplantation (Shaikhina et al. 2019).

The problem of learning optimal decision trees is an $\mathcal{NP}$-hard problem (Hyafil and Rivest 1976, Breiman et al. 1984). It can intuitively be viewed as a combinatorial optimization problem with an exponential number of decision variables: At each branching node of the tree, one can select which feature to branch on (and potentially the level of that feature), guiding each datapoint to the left or right using logical constraints.

1.1.1. Traditional Methods. Motivated by these hardness results, traditional algorithms for learning decision trees have relied on heuristics that use very intuitive, yet ad hoc, rules for constructing the decision trees. For example, Classification And Regression Tree uses the Gini Index to decide on the splitting (Breiman et al. 1984); Iterative Dichotomiser 3 uses entropy (Quinlan 1986); and C4.5 leverages normalized information gain (Quinlan 2014). The high quality and speed of these algorithms combined with the availability of software packages in many popular languages such as `R` or `Python` have facilitated their popularization (Kuhn et al. 2023a, b).

1.1.2. Mathematical Optimization Techniques. Motivated by the heuristic nature of traditional approaches that provide no guarantees on the quality of the learned tree, several researchers have proposed algorithms for learning provably *optimal* trees based on techniques from mathematical optimization. Approaches for learning optimal decision trees rely on enumeration coupled with rules to prune-out the search space. For example, Nijssen and Fromont (2010) use itemset mining algorithms and Narodytska et al. (2018) use satisfiability (SAT) solvers. Verhaeghe et al. (2020) propose a more elaborate implementation combining several ideas from the literature, including branch-and-bound, itemset mining techniques, and caching. Hu et al. (2019) use analytical bounds (to aggressively prune-out the search space) combined with a tailored bit-vector based implementation. Lin et al. (2020) extend the approach of Hu et al. (2019) to produce optimal decision trees over a variety of objectives such as F-score and area under the receiver operating characteristic curve (AUROC). Demirović et al. (2020) and Demirović and Stuckey (2021) use dynamic programming, and Aglin et al. (2020) use caching branch-and-bound search to compute optimal decision trees. Blanquero et al. (2021) propose a continuous-based randomized approach for learning optimal classification trees with oblique cuts. Incorporating constraints into the above approaches is usually a challenging task (Detassis et al. 2020).

1.1.3. Special Case of Mixed-Integer Optimization. As an alternative approach to conducting the search, Bertsimas and Dunn (2017) recently proposed using mixed-integer optimization (MIO) to learn optimal classification trees. Following this work, using MIO to learn decision trees gained a lot of traction in the literature with the works of Günlük et al. (2021), Aghaei et al. (2019), and Verwer and Zhang (2019). This is no coincidence. First, there exists a suite of MIO off-the-shelf solvers and algorithms that can be leveraged to effectively prune-out the search space. Indeed, solvers such as CPLEX (2009) and Gurobi (2015) have benefited from decades of research (Bixby 2012) and have been very successful at solving broad classes of MIO problems. Second, MIO comes with a highly expressive language that can be used to tailor the objective function of the problem or to augment the learning problem with constraints of practical interest. For example, Aghaei et al. (2019) leverage the power of MIO to learn fair and interpretable classification and regression trees by augmenting their model with additional constraints. They also show how MIO technology can be exploited to learn decision trees with more sophisticated structure (linear branching and leafing rules). Similarly, Günlük et al. (2021) use MIO to solve the problem of learning classification trees by taking into account the special structure of categorical features and allowing combinatorial decisions (based on subsets of values of features) at each node. MIO formulations have also been leveraged to design decision trees for decision- and policy-making problems (Azizi et al. 2018, Ciocan and Mišić 2022) for optimizing decisions over tree ensembles (Biggs and Hariss 2018, Mišić 2020) and for developing heuristic approaches for learning classification trees (Bertsimas and Dunn 2019). We refer the interested reader to Carrizosa et al. (2021) for an in-depth review of the field.

1.2. Discussion

The works of Bertsimas and Dunn (2017), Aghaei et al. (2019), and Verwer and Zhang (2019) have served to showcase the modeling power of using MIO to learn decision trees and the potential suboptimality of

traditional algorithms. Yet, we argue that they have not leveraged the power of MIO to its full extent.

A critical component for efficiently solving MIOs is to pose good formulations, but determining such formulations is no simple task. The standard approach for solving MIO problems is the branch-and-bound method, which partitions the search space recursively and solves linear optimization (LO) relaxations for each partition to produce bounds for fathoming sections of the search space. Thus, because solving an MIO requires solving a large sequence of LO problems, small and compact formulations are desirable as they enable the LO relaxation to be solved faster. Moreover, formulations with tight LO relaxations, referred to as *strong* formulations, are also desirable as they produce higher-quality bounds that lead to a faster pruning of the search space, ultimately reducing the number of LO problems to be solved. Indeed, a recent research thrust focuses on devising strong formulations for inference problems (Dong et al. 2015, Bienstock et al. 2018, Anderson et al. 2020, Bertsimas and Van Parys 2020, Xie and Deng 2020, Atamturk et al. 2021, Gómez 2021, Hazimeh et al. 2022). Unfortunately, these two goals are at odds with one another: Stronger formulations often require more variables and constraints than *weak* ones. For example, in the context of decision trees, Verwer and Zhang (2019) propose an MIO formulation with significantly fewer variables and constraints than the formulation of Bertsimas and Dunn (2017), but in the process weaken the LO relaxation. As a consequence, neither method consistently outperforms the other.

In the case of MIO problems with large numbers of decision variables and constraints, classical decomposition techniques from the operations research literature may be leveraged to break the problem up into multiple tractable subproblems of benign complexity (Gade et al. 2014, Liu et al. 2016, Liu and Sen 2020, Gangammanavar et al. 2021, Guo et al. 2021, MacNeil and Bodur 2022). A notable example of a decomposition algorithm is Benders' decomposition (Benders 1962). This decomposition approach exploits the structure of mathematical optimization problems with so-called *complicating variables* that couple constraints with one another and that, once fixed, result in an attractive decomposable structure that is leveraged to speed-up computation and alleviate memory consumption, allowing the solution of large-scale MIO problems.

To the best of our knowledge, existing approaches from the literature have not sought explicitly strong formulations and neither have they attempted to leverage the potentially decomposable structure of the problem. This is precisely the gap we fill with the present work.

1.3. Proposed Approach and Contributions

Our approach and main contributions in this paper are as follows:

(a) We propose a flow-based MIO formulation for learning optimal classification trees with binary features. In this model, correctly classified datapoints can be seen as *flowing* from the root of the tree to a suitable leaf while incorrectly classified datapoints are not allowed to flow through the tree. Our formulation can easily be augmented with constraints (e.g., imposing fairness), regularization penalties and conveniently be adjusted to cater for imbalanced data sets.

(b) We demonstrate that our proposed formulation has a stronger LO relaxation than existing alternatives. Notably, it does not involve big-M constraints. It is also amenable to Benders' decomposition. In particular, binary tests are selected in the main problem and each subproblem guides each datapoint through the tree via a max-flow subproblem. We leverage the max-flow structure of the subproblems to solve them efficiently via a tailored min-cut procedure. Moreover, we prove that all the constraints generated by this Benders' procedure are facet-defining; that is, they are required to describe the (convex hull of the) projection of the feasible region into the space of variables appearing in the main problem.

(c) We conduct extensive computational studies on benchmark data sets from the literature, showing that our formulations improve upon the state-of-the-art MIO algorithms, both in terms of in-sample solution quality (and speed) and out-of-sample performance.

The proposed modeling and solution paradigm can act as a building block for the faster and more accurate learning of more sophisticated trees and tree ensembles.

The rest of the paper is organized as follows. We introduce our flow-based formulation and our Benders' decomposition method in Section 2 and Section 3, respectively. Several generalizations of our core formulation are discussed in Section 4. We report on computational experiments with popular benchmark data sets in Section 5. Most proofs and detailed computational results are provided in the Online Appendix.

2. Learning Balanced Classification Trees

In this section, we describe our MIO formulation for learning optimal *balanced* classification trees of a given depth, that is, trees wherein the distance between all nodes where a prediction is made and the root node is equal to the tree depth. Our MIO formulation relies on the observation that once the structure of the tree is fixed, determining whether a datapoint is correctly classified or not reduces to checking whether the datapoint can, based on its features and label, flow from the root of the tree to a leaf where the prediction made matches its label. Thus, we begin this section by formally defining balanced trees and their associated *flow graph* in Section 2.1 before introducing our proposed flow-based formulation in Section 2.2. We discuss

variants of this basic model that can be used to learn sparse, possibly imbalanced, trees in Section 4. Throughout the paper, we represent vectors using bold fonts and sets using capital calligraphic fonts.

2.1. Decision Tree and Associated Flow Graph

A key step toward our flow-based MIO formulation of the problem consists in converting the decision tree of fixed depth that we wish to train to a directed acyclic graph where all arcs are directed from the root of the tree to the leaves. We detail this conversion together with the basic terminology that we use in our paper in what follows.

Definition 1 (Balanced Decision Tree). A balanced decision tree of depth $d \in \mathbb{N}$ is a perfect binary tree, that is, a binary tree in which all interior nodes have two children and all leaves have the same depth. We number the nodes in the tree in the same order they appear in the breadth-first search traverse such that 1 is the root node and $2^{d+1}-1$ is the bottom right node. We define the set of first 2^d-1 nodes $\mathcal{B} := \{1,\dots,2^d-1\}$ as *branching nodes* and the remaining 2^d nodes $\mathcal{L} := \{2^d,\dots,2^{d+1}-1\}$ as the *leaf nodes* of the decision tree.

An illustration of a balanced decision tree is provided in Figure 1 (left). The key idea behind our model is to convert a balanced decision tree to a *directed acyclic graph* by augmenting it with a single source node s that is connected to the root node (node 1) of the tree and a single sink node t connected to all leaf nodes of the tree. We refer to this graph as the *flow graph* of the decision tree. An illustration of these concepts on a decision tree of depth $d=2$ is provided on Figure 1.

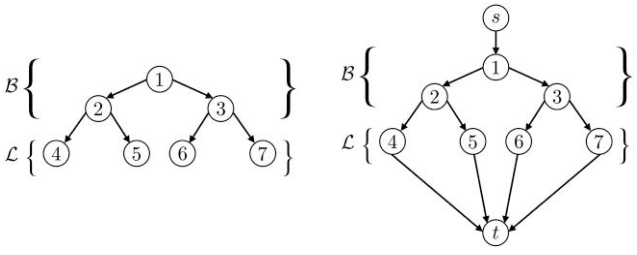
A formal definition for the flow graph associated with a balanced decision tree is as follows.

Definition 2 (Flow Graph of a Balanced Decision Tree). Given a balanced decision tree of depth d, define the directed *flow graph* $\mathcal{G}=(\mathcal{V},\mathcal{A})$ associated with the tree as follows. Let $\mathcal{V} := \{s,t\} \cup \mathcal{B} \cup \mathcal{L}$ be the vertices of the flow graph. Given $n \in \mathcal{B}$, let $\ell(n) := 2n$ be the *left descendant of* n, $r(n) := 2n+1$ be the *right descendant of* n, and

$$\mathcal{A} := \{(n,\ell(n)) : n \in \mathcal{B}\} \cup \{(n,r(n)) : n \in \mathcal{B}\} \cup \{(s,1)\} \cup \{(n,t) : n \in \mathcal{L}\}$$

be the arcs of the graph. Also, given $n \in \mathcal{B} \cup \mathcal{L}$, let $a(n)$ be the *ancestor* of n, defined through $a(1) := s$ and $a(n) := \lfloor n/2 \rfloor$ if $n \neq 1$.

Figure 1. Decision Tree of Depth 2 (Left) and Its Associated Flow Graph (Right)



Note. Here, $\mathcal{B}=\{1,2,3\}$ and $\mathcal{L}=\{4,5,6,7\}$, whereas $\mathcal{V}=\{s,1,2,\dots,7,t\}$ and $\mathcal{A}=\{(s,1),(1,2),\dots,(7,t)\}$.

2.2. Problem Formulation

Let $\mathcal{D} := \{\boldsymbol{x}^i, y^i\}_{i\in\mathcal{I}}$ be a training data set consisting of datapoints indexed in the set $\mathcal{I} \subseteq \mathbb{N}$. Each row $i \in \mathcal{I}$ consists of F binary features indexed in the set $\mathcal{F} \subseteq \mathbb{N}$, which we collect in the vector $\boldsymbol{x}^i \in \{0,1\}^F$, and a label y^i drawn from the finite set $\mathcal{K}$ of classes. In this section we formulate the problem of learning a multiclass classification tree of fixed finite depth $d \in \mathbb{N}$ that minimizes the misclassification rate (or equivalently, maximizes the number of correctly classified datapoints) based on MIO technology.

In our formulation, the classification tree is described through the branching variables $\boldsymbol{b}$ and the prediction variables $\boldsymbol{w}$. In particular, we use the variables $b_{nf} \in \{0,1\}, f \in \mathcal{F}, n \in \mathcal{B}$, to indicate if the tree branches on feature f at branching node n (i.e., it equals one if and only if the binary test performed at n asks "is $x^i_f = 0$"?). Accordingly, we use the variables $w^n_k \in \{0,1\}$, $n \in \mathcal{L}$, $k \in \mathcal{K}$ to indicate that at leaf node n the tree predicts class k. We use the auxiliary routing/flow variables $\boldsymbol{z}$ to decide on the flow of data through the flow graph associated with the decision tree. Specifically, for each node $n \in \mathcal{B} \cup \mathcal{L}$ and for each datapoint $i \in \mathcal{I}$, we introduce a decision variable $z^i_{a(n),n} \in \{0,1\}$ that equals one if and only if the ith datapoint is correctly classified and its flow traverses the arc $(a(n),n)$ on its way to the sink t. We let $z^i_{n,t}$ be defined accordingly for each arc between node $n \in \mathcal{L}$ and sink t. Datapoint $i \in \mathcal{I}$ is correctly classified if and only if its corresponding flow passes through some leaf node $n \in \mathcal{L}$ such that $w^n_{y_i} = 1$, that is, where the class predicted coincides with the class of the datapoint. If the flow of a datapoint i arrives at such a leaf node n and the datapoint is correctly classified, its corresponding flow is directed to the sink, that is, $z^i_{n,t} = 1$; otherwise, the corresponding flow is not initiated from the source at all. With these variables, the flow-based formulation reads

$$\text{maximize} \quad \sum_{i\in\mathcal{I}} \sum_{n\in\mathcal{L}} z^i_{n,t} \tag{1a}$$

$$\text{subject to} \quad \sum_{f\in\mathcal{F}} b_{nf} = 1 \qquad \forall n \in \mathcal{B}, \tag{1b}$$

$$z^i_{a(n),n} = z^i_{n,\ell(n)} + z^i_{n,r(n)} \qquad \forall n \in \mathcal{B}, i \in \mathcal{I}, \tag{1c}$$

$$z^i_{a(n),n} = z^i_{n,t} \qquad \forall n \in \mathcal{L}, i \in \mathcal{I}, \tag{1d}$$

$$z^i_{s,1} \leq 1 \qquad \forall i \in \mathcal{I}, \tag{1e}$$

$$z^i_{n,\ell(n)} \le \sum_{f\in\mathcal{F}:x^i_f=0} b_{nf} \qquad \forall n\in\mathcal{B}, i\in\mathcal{I}, \quad (1f)$$

$$z^i_{n,r(n)} \le \sum_{f\in\mathcal{F}:x^i_f=1} b_{nf} \qquad \forall n\in\mathcal{B}, i\in\mathcal{I}, \quad (1g)$$

$$z^i_{n,t} \le w^n_{y^i} \qquad \forall n\in\mathcal{L}, i\in\mathcal{I}, \quad (1h)$$

$$\sum_{k\in\mathcal{K}} w^n_k = 1 \qquad \forall n\in\mathcal{L}, \quad (1i)$$

$$w^n_k \in \{0,1\} \qquad \forall n\in\mathcal{L}, k\in\mathcal{K}, \quad (1j)$$

$$b_{nf} \in \{0,1\} \qquad \forall n\in\mathcal{B}, f\in\mathcal{F}, \quad (1k)$$

$$z^i_{a(n),n} \in \{0,1\} \qquad \forall n\in\mathcal{B}\cup\mathcal{L}, i\in\mathcal{I}, \quad (1l)$$

$$z^i_{n,t} \in \{0,1\} \qquad \forall n\in\mathcal{L}, i\in\mathcal{I}. \quad (1m)$$

An interpretation of the constraints and objective is as follows. Constraints (1b) ensure that at each branching node $n\in\mathcal{B}$ we branch on exactly one feature $f\in\mathcal{F}$. Constraints (1c) are flow conservation constraints for each datapoint i and node $n\in\mathcal{B}$: They ensure that if a datapoint arrives at a node, then it must also leave the node through one of its descendants. Similarly, Constraints (1d) enforce flow conservation for each node $n\in\mathcal{L}$. Inequality Constraints (1e) imply that at most one unit of flow can enter the graph through the source for each datapoint. Constraints (1f) (respectively, (1g)) ensure that if the flow of a datapoint is routed to the left (respectively, right) at node n, then one of the features such that $x^i_f = 0$ (respectively, $x^i_f = 1$) must have been selected for branching at the node. Constraints (1h) guarantee that datapoints whose flow is routed to the sink node t are correctly classified. Constraints (1i) make sure that each leaf node is assigned a predicted class $k\in\mathcal{K}$. Objective (1a) maximizes the total number of correctly classified datapoints.

Formulation (1) has several distinguishing features relative to existing MIO formulations for training decision trees. First, it does not use big-M constraints. Second, it includes *flow variables* indicating whether each datapoint is directed to the left or right at each branching node, which resembles the well-known multicommodity flow concept (Hu 1963). Third, incorrectly classified datapoints are associated with a flow of zero. In contrast, previous formulations (Bertsimas and Dunn 2017, Verwer and Zhang 2019) and more sophisticated formulations we propose later in Section 4 include binary decision variables that indicate the leaf each datapoint lands on. The advantage of modeling misclassified points with a flow of zero is that the resulting formulation is smaller. A similar idea of "projecting out" variables associated with misclassified datapoints was proposed by Günlük et al. (2021).

The number of variables and constraints in Problem (1) is $\mathcal{O}(2^d(|\mathcal{I}|+|\mathcal{F}|+|\mathcal{K}|))$, where d is the tree depth. Thus, its size is of the same order as the univariate splits formulation of Bertsimas and Dunn (2017) (formulation (24) in their paper) while being of higher order than that of Verwer and Zhang (2019). Nonetheless, the LO relaxation of Formulation (1) is tighter than those of Bertsimas and Dunn (2017) and Verwer and Zhang (2019), as demonstrated in the following theorem that we prove in Online Appendix EC.5, and therefore results in a more aggressive pruning of the branch-and-bound tree.

Theorem 1. *Problem* (1) *has a stronger LO relaxation than the formulations of Bertsimas and Dunn* (2017) *and Verwer and Zhang* (2019).

Remark 1. Formulation (1) assumes that all features $f\in\mathcal{F}$ are binary. However, this formulation can also be applied to data sets involving categorical or bounded integer features by first preprocessing the data as follows. For each categorical feature, we encode it as a one-hot vector; that is, for each level of the feature, we create a new binary column that has value 1 if and only if the original column has the corresponding level. We follow a similar approach for encoding integer features with a slight change. The new binary column has value 1 if and only if the main column has the corresponding value or any value smaller than it (Okada et al. 2019, Verwer and Zhang 2019). This discretization of the features increases the size of the data set linearly with the number of possible values of each categorical/integer feature. Formulation (1) only considers univariate splits. It can, however, be generalized to allow for multivariate splits. Indeed, in the case of binary features, having multivariate (or oblique) splits is equivalent to "combinatorial" branching, which can be done by creating additional artificial features (see Günlük et al. (2021) for additional discussion). We can also extend Formulation (1) for the case of nonbinary trees, where each splitting node can have more than two children. This can be achieved by introducing additional nodes and edges to the flow graph while adjusting the formulation accordingly.

3. Benders' Decomposition via Facet-Defining Cuts

In Section 2, we proposed a formulation for designing optimal classification trees that is provably stronger than existing approaches from the literature. Our model presents an attractive decomposable structure that we leverage in this section to speed-up computation.

3.1. Main Problem, Max-Flow Subproblems, and Benders' Decomposition

A classification tree is uniquely characterized by the branching decisions $\boldsymbol{b}$ and predictions $\boldsymbol{w}$ made at the

branching nodes and leaves, respectively. Given a choice of $\boldsymbol{b}$ and $\boldsymbol{w}$, each datapoint in Problem (1) is allotted one unit of flow that can be guided from the source node to the sink node through the flow graph associated with the decision tree. If the datapoint cannot be correctly classified, the flow that will reach the sink (and by extension enter the source) will be zero. In particular, once the branching variables $\boldsymbol{b}$ and prediction variables $\boldsymbol{w}$ have been fixed, optimization of the auxiliary flow variables $\boldsymbol{z}$ can be done separately for each datapoint. In particular, we can decompose Problem (1) into a *main problem* involving the variables $(\boldsymbol{b}, \boldsymbol{w})$ and $|\mathcal{I}|$ *subproblems* indexed by $i \in \mathcal{I}$ each involving the flow variables $\boldsymbol{z}^i$ associated with datapoint i. Additionally, each subproblem is a maximum flow problem for which specialized polynomial-time methods exist. Because of these characteristics, Formulation (1) can be naturally tackled using Benders' decomposition (Benders 1962). In what follows, we describe the Benders' decomposition approach.

Problem (1) can be written equivalently as

$$\text{maximize} \quad \sum_{i\in\mathcal{I}} g^i(\boldsymbol{b},\boldsymbol{w}) \tag{2a}$$

$$\text{subject to} \quad \sum_{f\in\mathcal{F}} b_{nf} = 1 \qquad \forall n \in \mathcal{B}, \tag{2b}$$

$$\sum_{k\in\mathcal{K}} w^n_k = 1 \qquad \forall n \in \mathcal{L}, \tag{2c}$$

$$b_{nf} \in \{0,1\} \qquad \forall n \in \mathcal{B}, f \in \mathcal{F}, \tag{2d}$$

$$w^n_k \in \{0,1\} \qquad \forall n \in \mathcal{L}, k \in \mathcal{K}, \tag{2e}$$

where, for any fixed $i \in \mathcal{I}$, $\boldsymbol{w}$ and $\boldsymbol{b}$, the quantity $g^i(\boldsymbol{b},\boldsymbol{w})$ is defined as the optimal objective value of the problem

$$g^i(\boldsymbol{b},\boldsymbol{w}) = \text{maximize} \quad \sum_{n\in\mathcal{L}} z^i_{n,t} \tag{3a}$$

$$\text{subject to} \quad z^i_{a(n),n} = z^i_{n,\ell(n)} + z^i_{n,r(n)} \qquad \forall n \in \mathcal{B}, \tag{3b}$$

$$z^i_{a(n),n} = z^i_{n,t} \qquad \forall n \in \mathcal{L}, \tag{3c}$$

$$z^i_{s,1} \le 1 \tag{3d}$$

$$z^i_{n,\ell(n)} \le \sum_{f\in\mathcal{F}:x^i_f=0} b_{nf} \qquad \forall n \in \mathcal{B}, \tag{3e}$$

$$z^i_{n,r(n)} \le \sum_{f\in\mathcal{F}:x^i_f=1} b_{nf} \qquad \forall n \in \mathcal{B}, \tag{3f}$$

$$z^i_{n,t} \le w^n_{y^i} \qquad \forall n \in \mathcal{L}, \tag{3g}$$

$$z^i_{a(n),n} \in \{0,1\} \qquad \forall n \in \mathcal{B}\cup\mathcal{L}, \tag{3h}$$

$$z^i_{n,t} \in \{0,1\} \qquad \forall n \in \mathcal{L}. \tag{3i}$$

Problem (3) is a maximum flow problem on the flow graph $\mathcal{G}$ (see Definition 2) whose arc capacities are determined by $(\boldsymbol{b},\boldsymbol{w})$ and datapoint $i \in \mathcal{I}$, as formalized next.

Definition 3 (Capacitated Flow Graph). Given the flow graph $\mathcal{G} = (\mathcal{V},\mathcal{A})$, vectors $(\boldsymbol{b},\boldsymbol{w})$, and datapoint $i \in \mathcal{I}$, define arc capacities $c^i(\boldsymbol{b},\boldsymbol{w})$ as follows. Let $c^i_{s,1}(\boldsymbol{b},\boldsymbol{w}) := 1$, $c^i_{n,\ell(n)}(\boldsymbol{b},\boldsymbol{w}) := \sum_{f\in\mathcal{F}:x^i_f=0} b_{nf}$ and $c^i_{n,r(n)}(\boldsymbol{b},\boldsymbol{w}) := \sum_{f\in\mathcal{F}:x^i_f=1} b_{nf}$ for all $n \in \mathcal{B}$, and $c^i_{n,t}(\boldsymbol{b},\boldsymbol{w}) := w^n_{y^i}$ for $n \in \mathcal{L}$. Define the *capacitated flow graph* $\mathcal{G}^i(\boldsymbol{b},\boldsymbol{w})$ as the flow graph $\mathcal{G}$ augmented with capacities $c^i(\boldsymbol{b},\boldsymbol{w})$.

Observe that the arc capacities in $\mathcal{G}^i(\boldsymbol{b},\boldsymbol{w})$ are affine functions of $(\boldsymbol{b},\boldsymbol{w})$. An interpretation of the arc capacities and capacitated flow graph for $\boldsymbol{b}$ and $\boldsymbol{w}$ feasible in Problem (2) is as follows. Constraints (2b) imply that the weights $c^i(\boldsymbol{b},\boldsymbol{w})$, $i \in \mathcal{I}$, in the previous definition are binary. Indeed, the capacity of the arc incoming into node 1 is one. Moreover, for each node $n \in \mathcal{B}$ and datapoint $i \in \mathcal{I}$, exactly one of the outgoing arcs of node n in graph $\mathcal{G}^i(\boldsymbol{b},\boldsymbol{w})$ has capacity 1: The left arc if the datapoint passes the test ($c^i_{n,\ell(n)}(\boldsymbol{b},\boldsymbol{w}) = 1$) or the right arc if it fails it ($c^i_{n,r(n)}(\boldsymbol{b},\boldsymbol{w}) = 1$). Finally, for each node $n \in \mathcal{L}$ in graph $\mathcal{G}^i(\boldsymbol{b},\boldsymbol{w})$, its outgoing arc has capacity one if and only if the datapoint has the same label y^i as that predicted at node n. Thus, for each datapoint, the set of arcs with capacity 1 in the graph $\mathcal{G}^i(\boldsymbol{b},\boldsymbol{w})$ forms a path from the source node to the leaf where this datapoint is assigned in the decision tree. This path is connected to the sink via an arc of capacity 1 if and only if the datapoint is correctly classified. As all the arc capacities in the flow graph are binary, integrality Constraints (3h) and (3i) can be relaxed.

Problem (3) is equivalent to a maximum flow problem on the capacitated flow graph $\mathcal{G}^i(\boldsymbol{b},\boldsymbol{w})$. From the well-known max-flow/min-cut duality (Vazirani 2013), it follows that $g^i(\boldsymbol{b},\boldsymbol{w})$ is the capacity of a minimum (s,t) cut of graph $\mathcal{G}^i(\boldsymbol{b},\boldsymbol{w})$. In other words, it is the greatest value smaller than or equal to the value of all cuts in graph $\mathcal{G}^i(\boldsymbol{b},\boldsymbol{w})$. Given a set $\mathcal{S} \subseteq \mathcal{V}$, we let $\mathcal{C}(\mathcal{S}) := \{(n_1,n_2) \in \mathcal{A} : n_1 \in \mathcal{S}, n_2 \notin \mathcal{S}\}$ denote the cut-set corresponding to the source set $\mathcal{S}$. With this notation, Problem (2) can be reformulated as

$$\underset{\boldsymbol{g},\boldsymbol{b},\boldsymbol{w}}{\text{maximize}} \quad \sum_{i\in\mathcal{I}} g^i \tag{4a}$$

$$\text{subject to} \quad g^i \le \sum_{(n_1,n_2)\in\mathcal{C}(\mathcal{S})} c^i_{n_1,n_2}(\boldsymbol{b},\boldsymbol{w}) \qquad \forall i \in \mathcal{I}, \mathcal{S} \subseteq \mathcal{V}\setminus\{t\} : s \in \mathcal{S}, \tag{4b}$$

$$\sum_{f\in\mathcal{F}} b_{nf} = 1 \qquad \forall n \in \mathcal{B}, \tag{4c}$$

$$\sum_{k\in\mathcal{K}} w^n_k = 1 \qquad \forall n \in \mathcal{L}, \tag{4d}$$

$$b_{nf} \in \{0,1\} \qquad \forall n \in \mathcal{B}, f \in \mathcal{F}, \tag{4e}$$

$$w^n_k \in \{0,1\} \qquad \forall n \in \mathcal{L}, k \in \mathcal{K}, \tag{4f}$$

$$g^i \le 1 \qquad \forall i \in \mathcal{I}, \tag{4g}$$

where, at an optimal solution, g^i represents the value of a minimum (s, t) cut of graph $\mathcal{G}^i(\boldsymbol{b}, \boldsymbol{w})$. Indeed, Objective Function (4a) maximizes the g^i, whereas Constraints (4b) ensure that the value of g^i is no greater than the value of a minimum cut.

Formulation (4) contains an exponential number of Inequalities (4b) and is implemented using row generation, whereby Constraints (4b) are initially dropped and added on the fly during optimization as Benders' cuts. We added the redundant constraints (4g) to ensure Problem (4) is bounded even if all Constraints (4b) are dropped. Row generation can be implemented in modern MIO solvers via callbacks by adding lazy constraints at relevant nodes of the branch-and-bound tree. Identifying which Constraint (4b) to add can in general be done by solving a minimum cut problem, and could in principle be solved via well-known algorithms, such as Goldberg and Tarjan (1988) and Hochbaum (2008).

The Benders' method we propose is reminiscent of the approach proposed by Lozano and Smith (2022) to tackle two-stage problems in which the subproblem corresponds to a "tractable" 0-1 problem (i.e., a problem that admits an exact LO reformulation). In Lozano and Smith (2022), this subproblem is a decision diagram; in our case, it is a maximum flow problem. Thus, Inequalities (4b) correspond to usual Benders' cuts that replace the discrete subproblem with its (equivalent) continuous LO reformulation. As also pointed out by Lozano and Smith (2022), at each iteration of Benders' method, there are often multiple optimal dual solutions, each corresponding to a candidate inequality that can be added. Cuts obtained from most of these solutions might be weak in general, and thus they discuss how to strengthen them. In the next section, we propose a method that, among all potential optimal dual solutions, finds one that is guaranteed to result in a facet-defining cut for the convex hull of the feasible region given by Inequalities (4b)–(4g), that is, a cut that is already the best possible and admits no further strengthening. In addition, the method is faster than using a general purpose method to find minimum cuts or solve LO problems, which we show in Online Appendix EC.8.1.

3.2. Generating Facet-Defining Cuts via a Tailored Min-Cut Procedure

Row generation methods for integer optimization problems such as Benders' decomposition may require a long time to converge to an optimal solution if each added inequality is weak for the feasible region of interest. It is therefore of critical importance to add strong nondominated cuts at each iteration of the algorithm (Magnanti and Wong 1981). We now argue that not all Inequalities (4b) are facet-defining for the convex hull of the feasible set of Problem (4). To this end, let $\mathcal{H}_=$ represent the (mixed-binary) feasible region defined by Constraints (4b)–(4g) and denote by $\text{conv}(\mathcal{H}_=)$ its convex hull. Example 1 shows that some of Inequalities (4b) are not facet-defining for $\text{conv}(\mathcal{H}_=)$, even if they correspond to a minimum cut for a given value of $(\boldsymbol{b}, \boldsymbol{w})$, and are in fact dominated.

Example 1. Consider an instance of Problem (4) with a depth $d = 1$ decision tree (i.e., $\mathcal{B} = \{1\}$ and $\mathcal{L} = \{2, 3\}$) and a data set involving a single feature ($\mathcal{F} = \{1\}$). Consider datapoint i such that $x_1^i = 0$ and $y^i = 1$. Suppose that the solution to the main problem is such that we branch on (the unique) feature at node 1 and predict class 0 at nodes 2 and 3. Then, datapoint i is routed left at node 1 and is misclassified. A valid min-cut for the resulting graph includes all arcs incoming into the sink, that is, $\mathcal{S} = \{s, 1, 2, 3\}$ and $\mathcal{C}(\mathcal{S}) = \{(2, t), (3, t)\}$. The associated Benders' inequality (4b) reads

$$g^i \leq w_1^2 + w_1^3. \tag{5}$$

Intuitively, (5) states that datapoint i can be correctly classified if its class label is assigned to at least one node and is certainly valid for $\text{conv}(\mathcal{H}_=)$. However, because datapoint i cannot be routed to node 3, the stronger inequality

$$g^i \leq w_1^2 \tag{6}$$

is valid for $\text{conv}(\mathcal{H}_=)$ and dominates (5).

Example 1 implies that an implementation of Formulation (4) using general purpose min-cut algorithms to identify constraints to add may perform poorly. This motivates us to develop a tailored algorithm that exploits the structure of each capacitated flow graph $\mathcal{G}^i(\boldsymbol{b}, \boldsymbol{w})$ to return inequalities that are never dominated, thus resulting in faster convergence of the Benders' decomposition approach.

Algorithm 1 shows the proposed procedure, which can be called at *integer nodes* of the branch-and-bound tree, using for example callback procedures available in most commercial and open-source solvers. Algorithm 1 traverses the flow graph associated with a datapoint using depth-first search to investigate the existence of a path from the source to the sink. If there is such a path, it means that the datapoint is correctly classified and there are no violating inequalities. Otherwise, it outputs the set of all observed nodes in the traverse as the source set of the min-cut. Figure 2 illustrates Algorithm 1. We now prove that Algorithm 1 is indeed a valid *separation algorithm*.

Algorithm 1 (Separation Procedure)

Input: $(\boldsymbol{b}, \boldsymbol{w}, \boldsymbol{g}) \in \{0,1\}^{\mathcal{B}\times\mathcal{F}} \times \{0,1\}^{\mathcal{L}\times\mathcal{K}} \times \mathbb{R}^{\mathcal{I}}$ satisfying (4c)–(4g); $i \in \mathcal{I}$: data point used to generate the cut.
Output: -1 if all Constraints (4b) corresponding to i are satisfied; source set $\mathcal{S}$ of min-cut otherwise.

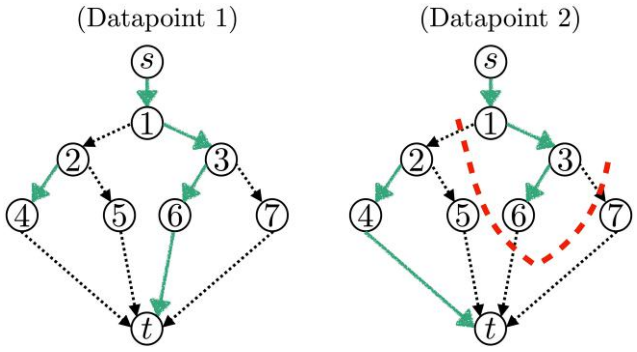
1: **if** $g^i = 0$ **then return** -1
2: **Initialize** $n \leftarrow 1$ ▷ Current node = root
3: **Initialize** $\mathcal{S} \leftarrow \{s\}$ ▷ $\mathcal{S}$ is in the source set of the cut
4: **while** $n \in \mathcal{B}$ **do**
5: $\mathcal{S} \leftarrow \mathcal{S} \cup \{n\}$
6: **if** $c^i_{n,\ell(n)}(\boldsymbol{b},\boldsymbol{w}) = 1$ **then**
7: $n \leftarrow \ell(n)$ ▷ Datapoint i is routed left
8: **else if** $c^i_{n,r(n)}(\boldsymbol{b},\boldsymbol{w}) = 1$ **then**
9: $n \leftarrow r(n)$ ▷ Datapoint i is routed right
10: **end if**
11: **end while** ▷ At this point, $n \in \mathcal{L}$
12: $\mathcal{S} \leftarrow \mathcal{S} \cup \{n\}$
13: **if** $g^i > c^i_{n,t}(\boldsymbol{b},\boldsymbol{w})$ **then**
▷ Minimum cut $\mathcal{S}$ with capacity 0 found
14: **return** $\mathcal{S}$
15: **else** ▷ Minimum cut $\mathcal{S}$ has capacity 1, Constraints (4b) are satisfied
16: **return** -1
17: **end if**

Proposition 1. *Given $i \in \mathcal{I}$ and $(\boldsymbol{b},\boldsymbol{w},\boldsymbol{g})$ satisfying (4c)–(4g) (in particular, $\boldsymbol{b}$ and $\boldsymbol{w}$ are integral), Algorithm 1 either finds a violated inequality (4b) or proves that all such inequalities are satisfied.*

Proof. The right-hand side of (4b), which corresponds to the capacity of a cut in the graph, is nonnegative. Therefore, if $g^i = 0$ (line 1), all inequalities are automatically satisfied. Because $(\boldsymbol{b},\boldsymbol{w})$ is integer, all arc capacities are either zero or one. We assume that the arcs with zero capacity are removed from the flow graph. Moreover, because $g^i \le 1$, we find that either the value of a minimum cut is zero, and there exists a violated inequality or the value of a minimum cut is at least one, and there is no violated inequality. Finally, there exists a 0-capacity cut if and only if s and t belong to different connected components in the graph $\mathcal{G}^i(\boldsymbol{b},\boldsymbol{w})$.

The connected component s belongs to, can be found using depth-first search. For any fixed $n \in \mathcal{B}$, Constraints (4c) and the definition of $c^i(\boldsymbol{b},\boldsymbol{w})$ imply that either arc $(n,\ell(n))$ or arc $(n,r(n))$ has capacity 1 (but not both). If arc $(n,\ell(n))$ has capacity 1 (line 6), then $\ell(n)$ can be added to the component connected to s (set $\mathcal{S}$); the case where arc $(n,r(n))$ has capacity 1 (line 8) is handled analogously. This process continues until a leaf node is reached (line 12). If the capacity of the arc to the sink is 1 (line 15), then an $s-t$ path is found and no cut with capacity 0 exists. Otherwise (line 13), $\mathcal{S}$ is the connected component of s and $t \notin \mathcal{S}$, thus $\mathcal{S}$ is the source of a minimum cut with capacity 0. □

Figure 2. (Color online) Algorithm 1 on Two Datapoints That Are Correctly Classified (Datapoint 1, Left) and Incorrectly Classified (Datapoint 2, Right)



Notes. Unbroken arcs (n,n') have capacity $c^i_{n,n'}(\boldsymbol{b},\boldsymbol{w}) = 1$ (and others capacity 0). In the case of datapoint 1, which is correctly classified because there exists a path from source to sink, Algorithm 1 terminates on line 16 and returns -1. In the case of datapoint 2, which is incorrectly classified, Algorithm 1 returns set $\mathcal{S} = \{s,1,3,6\}$ on line 14. The associated minimum cut consists of arcs (1, 2), $(6,t)$, and (3, 7) and is represented by the thick dashed line.

Observe that Algorithm 1, which exploits the specific structure of the network for binary $(\boldsymbol{b},\boldsymbol{w})$ feasible in (2), is much faster than general purpose minimum-cut methods. Indeed, because at each iteration in the main loop (lines 4–11), the value of n is updated to a descendant of n, the algorithm terminates in at most $\mathcal{O}(d)$ iterations, where d is the depth of the tree. As $|\mathcal{B} \cup \mathcal{L}|$ is $\mathcal{O}(2^d)$, the complexity is logarithmic in the size of the tree. In addition to providing a very fast method for generating Benders' inequalities at integer nodes of a branch-and-bound tree, Algorithm 1 is guaranteed to generate strong nondominated inequalities.

Define

$$\mathcal{H}_{\le} := \left\{ \begin{array}{c} \boldsymbol{b} \in \{0,1\}^{\mathcal{B}\times\mathcal{F}}, \boldsymbol{w} \in \{0,1\}^{\mathcal{L}\times\mathcal{K}}, \boldsymbol{g} \in \mathbb{R}^{\mathcal{I}} : \\ \sum_{f\in\mathcal{F}} b_{nf} \le 1 \quad \forall n \in \mathcal{B} \\ \sum_{k\in\mathcal{K}} w^n_k \le 1 \quad \forall n \in \mathcal{L} \\ g^i \le \sum_{(n_1,n_2)\in\mathcal{C}(\mathcal{S})} c^i_{n_1,n_2}(\boldsymbol{b},\boldsymbol{w}) \\ \forall i \in \mathcal{I}, \mathcal{S} \subseteq \mathcal{V} \setminus \{t\} : s \in \mathcal{S} \end{array} \right\}.$$

Our main motivation for introducing $\mathcal{H}_{\le}$ is that $\text{conv}(\mathcal{H}_{\le})$ is full dimensional, whereas $\text{conv}(\mathcal{H}_{=})$ is not. Moreover, in Formulation (4), apart from Constraints (4c)–(4d), variables $\boldsymbol{b}$ and $\boldsymbol{w}$ only appear in the right-hand side of Inequalities (4b) with nonnegative coefficients. Therefore, replacing Constraints (4c)–(4f) with $(\boldsymbol{b},\boldsymbol{w}) \in \mathcal{H}_{\le}$ still results in valid formulation for Problem (4), because there exists an optimal solution where the inequalities are tight. Theorem 2 formally states that Algorithm 1 generates nondominated inequalities.

Theorem 2. *All violated inequalities generated by Algorithm 1 are facet-defining for* $\text{conv}(\mathcal{H}_{\le})$.

Example 2 (Example 1 Continued). In the instance considered in Example 1, if $b_{1f} = 1$ and $w_1^2 = 0$, then the cut generated by Algorithm 1 has source set $\mathcal{S} = \{s, 1, 2\}$ and results in inequality

$$g^i \leq \sum_{(n_1,n_2)\in\mathcal{C}(\mathcal{S})} c^i_{n_1,n_2}(\boldsymbol{b},\boldsymbol{w}) = c^i_{2,4}(\boldsymbol{b},\boldsymbol{w}) + c^i_{1,3}(\boldsymbol{b},\boldsymbol{w}) = w_1^2,$$

which is precisely the stronger inequality (6).

Remark 2. Algorithm 1 can only be invoked at integer nodes of the branch-and-bound tree. However, there is a slight advantage in including as many cut-set inequalities as possible at the root node of the branch-and-bound tree by using Gurobi to solve the corresponding linear optimization problem for each subproblem. This approach allows us to enhance the LO relaxation at integer nodes. A detailed investigation on this matter is reported in Online Appendix EC.8.1.

4. Generalizations

In Section 2, we proposed a flow-based MIO formulation for designing optimal *balanced* decision trees (Problem (1)). In this section, we generalize this core formulation to design regularized (i.e., not necessarily balanced) classification trees wherein the distance from root to leaf may vary across leaves (Section 4.1). We also discuss a variant that tracks all datapoints, even those that are not correctly classified, making it suitable to learn from imbalanced data sets (Section 4.2) and to design fair decision trees (Online Appendix EC.1).

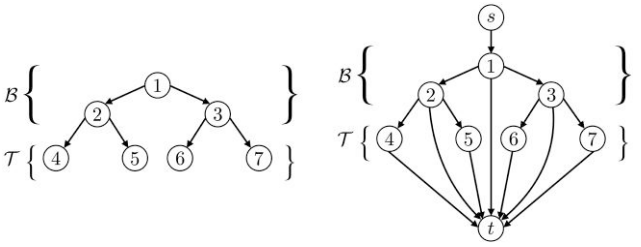
4.1. Imbalanced Decision Trees

Formulation (1) outputs a balanced decision tree as defined in Definition 1. Such trees may result in overfitting of the data and poor out-of-sample performance, in particular if d is large. To this end, we propose a variant of Formulation (1), which allows for the design of trees that are not necessarily balanced and that have the potential of performing better out-of-sample. To this end, we introduce the following terminology pertaining to *imbalanced* trees.

Definition 4 (Imbalanced Decision Trees). An imbalanced decision tree of (maximum) depth $d \in \mathbb{N}$ is a full binary tree, that is, a tree in which every node has either zero or two children and where the largest depth of a leaf is d. We let $\mathcal{B} := \{1, \ldots, 2^d - 1\}$ denote the set of all *candidate* branching nodes and $\mathcal{T} := \{2^d, \ldots, 2^{d+1} - 1\}$ represent the set of *terminal nodes*. We will refer to a node $n \in \mathcal{B} \cup \mathcal{T}$ as a *leaf* if no branching occurs at the node.

In a *balanced* decision tree, see Definition 1, branching occurs at all nodes in $\mathcal{B}$, and leaves of the decision tree correspond precisely to all nodes of maximum depth, that is, $\mathcal{L} = \mathcal{T}$. In contrast, in *imbalanced* decision trees nodes in $\mathcal{B}$ can be leaf nodes and terminal nodes in $\mathcal{T}$ need not be part of the decision tree.

Figure 3. Decision Tree of Depth 2 (Left) and Its Associated Flow Graph (Right) That Can Be Used to Train Imbalanced Decision Trees of Maximum Depth 2



Notes. Here, $\mathcal{B} = \{1,2,3\}$ and $\mathcal{T} = \{4,5,6,7\}$, whereas $\mathcal{V} = \{s, 1, 2, \ldots, 7, t\}$ and $\mathcal{A} = \{(s,1),(1,2),(1,t),\ldots,(7,t)\}$. The additional arcs that connect the branching nodes to the sink allow branching nodes $n \in \mathcal{B}$ to be converted to leaves where a prediction is made. Correctly classified datapoints that reach a leaf are directed to the sink. Incorrectly classified datapoints are not allowed to flow in the graph.

Akin to Definition 2, we associate with an imbalanced decision tree a directed acyclic graph by augmenting the tree with a single source node s that is connected to the root node of the tree and a single sink node t that is connected to *all nodes* $n \in \mathcal{B} \cup \mathcal{T}$ of the tree, allowing correctly classified datapoints to flow to the sink from any node where a prediction is made (leaf of the learned tree). An illustration of these concepts on an imbalanced decision tree of depth $d = 2$ is provided on Figure 3. A formal definition for the flow graph associated with a imbalanced decision tree of maximum depth d is as follows.

Definition 5 (Flow Graph of an Imbalanced Decision Tree). Given an imbalanced decision tree of depth d, we define its associated directed *flow graph* $\mathcal{G} = (\mathcal{V}, \mathcal{A})$ as follows. Let $\mathcal{V} := \{s, t\} \cup \mathcal{B} \cup \mathcal{T}$ be the vertices of the flow graph. Given $n \in \mathcal{B}$, let $\ell(n) := 2n$ be the *left descendant of* n, $r(n) := 2n + 1$ be the *right descendant of* n, and

$$\mathcal{A} := \{(n, \ell(n)) : n \in \mathcal{B}\} \cup \{(n, r(n)) : n \in \mathcal{B}\} \cup \{(s,1)\} \cup \{(n,t) : n \in \mathcal{B} \cup \mathcal{T}\}$$

be the arcs of the graph. Also, given $n \in \mathcal{B} \cup \mathcal{T}$, let $a(n)$ be the *parent* of n, defined through $a(1) := s$ and $a(n) := \lfloor n/2 \rfloor$ if $n \neq 1$.

We are now ready to formulate the variant of Problem (1) that allows for the design of imbalanced classification trees. In addition to the decision variables from Formulation (1), we introduce, for every node $n \in \mathcal{B} \cup \mathcal{T}$, the binary decision variable p_n that has a value of one if and only if node n is a leaf node of the tree, that is, if we make a prediction at node n. The auxiliary routing/flow variables z now account for all arcs in the flow graph introduced in Definition 5. The

problem of learning optimal imbalanced classification trees is then expressible as

$$\text{maximize} \quad (1-\lambda)\sum_{i\in\mathcal{I}}\sum_{n\in\mathcal{B}\cup\mathcal{T}} z^i_{n,t} - \lambda\sum_{n\in\mathcal{B}}\sum_{f\in\mathcal{F}} b_{nf} \tag{7a}$$

$$\text{subject to} \quad \sum_{f\in\mathcal{F}} b_{nf} + p_n + \sum_{m\in\mathcal{P}(n)} p_m = 1 \qquad \forall n\in\mathcal{B}, \tag{7b}$$

$$p_n + \sum_{m\in\mathcal{P}(n)} p_m = 1 \qquad \forall n\in\mathcal{T}, \tag{7c}$$

$$z^i_{a(n),n} = z^i_{n,\ell(n)} + z^i_{n,r(n)} + z^i_{n,t} \qquad \forall n\in\mathcal{B}, i\in\mathcal{I}, \tag{7d}$$

$$z^i_{a(n),n} = z^i_{n,t} \qquad \forall n\in\mathcal{T}, i\in\mathcal{I}, \tag{7e}$$

$$z^i_{s,1} \le 1 \qquad \forall i\in\mathcal{I}, \tag{7f}$$

$$z^i_{n,\ell(n)} \le \sum_{f\in\mathcal{F}:x^i_f=0} b_{nf} \qquad \forall n\in\mathcal{B}, i\in\mathcal{I}, \tag{7g}$$

$$z^i_{n,r(n)} \le \sum_{f\in\mathcal{F}:x^i_f=1} b_{nf} \qquad \forall n\in\mathcal{B}, i\in\mathcal{I}, \tag{7h}$$

$$z^i_{n,t} \le w^n_{y^i} \qquad \forall n\in\mathcal{B}\cup\mathcal{T}, i\in\mathcal{I}, \tag{7i}$$

$$\sum_{k\in\mathcal{K}} w^n_k = p_n \qquad \forall n\in\mathcal{B}\cup\mathcal{T}, \tag{7j}$$

$$w^n_k \in \{0,1\} \qquad \forall n\in\mathcal{B}\cup\mathcal{T}, k\in\mathcal{K}, \tag{7k}$$

$$b_{nf} \in \{0,1\} \qquad \forall n\in\mathcal{B}, f\in\mathcal{F}, \tag{7l}$$

$$p_n \in \{0,1\} \qquad \forall n\in\mathcal{B}\cup\mathcal{T}, \tag{7m}$$

$$z^i_{a(n),n}, z^i_{n,t} \in \{0,1\} \qquad \forall n\in\mathcal{B}\cup\mathcal{T}, i\in\mathcal{I}, \tag{7n}$$

where $\mathcal{P}(n)$ is the set of all ancestors of node $n\in\mathcal{B}\cup\mathcal{T}$, that is, the set of all nodes lying on the unique path from the root node to node n, and $\lambda\in[0,1]$ is a regularization parameter. An explanation of the new/modified problem constraints is as follows. Constraints (7b) imply that at any node $n\in\mathcal{B}$ we either branch on a feature f (if $\sum_{f\in\mathcal{F}} b_{nf}=1$), predict a label (if $p_n=1$), or get pruned if a prediction is made at one of the node ancestors (i.e., if $\sum_{m\in\mathcal{P}(n)} p_m = 1$). Similarly Constraints (7c) ensure that any node $n\in\mathcal{T}$ is either a leaf node of the tree or is pruned. Constraints (7d)–(7j) exactly mirror Constraints (1c)–(1i) in Problem (1). They are slight modifications of the original constraints accounting for the new arcs added to the flow graph and for the possibility of making predictions at branching nodes $n\in\mathcal{B}$. In particular, Constraints (7j) imply that if a node n gets pruned we do not predict any class at the node, that is, $w^n_k=0$ for all $k\in\mathcal{K}$. A penalty term is added to (7a) to encourage sparser trees with fewer branching decisions. Although it is feasible to design a decision tree with the same branching decisions in multiple nodes on a single path from root to sink, such solutions are never optimal for (7) if $\lambda>0$, as a simpler tree would result in the same misclassification.

Problem (7) allows for the design of regularized decision trees by augmenting this nominal formulation with additional regularization constraints. These either limit or penalize the number of nodes that can be used for branching or place a lower bound on the number of datapoints that land on each leaf. We detail these variants in the following. All these variants either explicitly or implicitly restrict the tree size, thereby mitigating the risk of overfitting and resulting in more interpretable trees.

4.1.1. Sparsity.

A commonly used approach to restrict tree size is to add sparsity constraints that restrict the number of branching nodes (Breiman et al. 1984, Quinlan 2014, Bertsimas and Dunn 2017, Aghaei et al. 2019, Blanquero et al. 2020). Such constraints are usually met in traditional methods by performing a postprocessing *pruning* step on the learned (unrestricted) tree. We enforce this restriction by adding the constraint

$$\sum_{n\in\mathcal{B}}\sum_{f\in\mathcal{F}} b_{nf} \le C \tag{8}$$

to Problem (7) and possibly setting $\lambda=0$. This constraint ensures that the learned tree has at most C branching nodes (thus at most $C+1$ leaf nodes), where C is a hyperparameter that can be tuned using cross-validation (Bishop 2006). Because of the nonconvexity introduced by the integer variables in Problem (7), the constrained sparsity formulation provides more options than the penalized version (Lombardi et al. 2020). In other words, for any choice of $\lambda\in[0,1]$ in the penalized version, there exists a choice of $C\in\{0,\dots,2^d\}$ in the constrained version that yields the same solution, but the converse is not necessarily true. The penalized version is more common in the machine learning literature and we will thus use it for benchmarking purposes in our numerical results (Section 5).

4.1.2. Maximum Number of Features to Use.

One can also constrain the total number C of features branched on in the decision tree as in Aghaei et al. (2019) by adding the constraints

$$\sum_{f\in\mathcal{F}} b_f \le C \text{ and } b_f \ge b_{nf} \quad \forall n\in\mathcal{B}, f\in\mathcal{F} \tag{9}$$

to Problem (7), where $b_f\in\{0,1\}, f\in\mathcal{F}$ are decision variables that indicate if feature f is used in the tree. Parameter C can be a user-specified input or tuned via cross-validation. Neither integrality nor bound constraints on variable b_f need to be enforced in the formulation.

4.1.3. Minimum Number of Datapoints in Each Leaf Node.

Another popular regularization approach (Bertsimas and Dunn 2017) consists in placing a lower bound on the number of datapoints that land in each leaf. To ensure that each node contains at least $N_{\min}$ datapoints, we impose

$$\sum_{i\in\mathcal{I}} z^i_{a(n),n} \geq N_{\min} p_n \quad \forall n \in \mathcal{B}\cup\mathcal{T} \tag{10}$$

in Problem (7).

4.2. Imbalanced Data Sets

A data set is called *imbalanced* when the class distribution is not uniform, that is, when the number of datapoints in each class varies significantly from class to class. In the case when a data set consists of two classes, that is, $\mathcal{K} := \{k, k'\}$, we say that it is imbalanced if

$$|\{i \in \mathcal{I} : y^i = k\}| \gg |\{i \in \mathcal{I} : y^i = k'\}|,$$

in which case k and k' are referred to as the majority and minority classes, respectively. In imbalanced data sets, predicting the majority class for all datapoints results in high accuracy, and thus decision trees that maximize prediction accuracy without accounting for the imbalanced nature of the data perform poorly on the minority class. Imbalanced data sets occur in many important domains, for example, to predict landslides, to detect fraud, or to predict if a patient has cancer (Kirschbaum et al. 2009, Khalilia et al. 2011, Wei et al. 2013). Naturally, being able to predict the minority class(es) accurately is crucial in such settings.

In the following we propose adjustments to the core formulation (7) to ensure meaningful decision trees are learned even in the case of imbalanced data sets. These adjustments require calculating metrics such as true positives, true negatives, false positives, and false negatives or some function of them such as recall (fraction of all datapoints from the positive class that are correctly identified) and precision (the portion of correctly classified datapoints from the positive class out of all datapoints with positive predicted class). Thus, learning meaningful decision trees requires us to track *all datapoints*, not only the correctly classified ones as done in formulations (1) and (7).

To this end, we propose to replace the single sink in the flow graph associated with a imbalanced decision tree, see Definition 5, with $|\mathcal{K}|$ sink nodes denoted by t_k, one for each class $k \in \mathcal{K}$. Each sink node t_k, $k \in \mathcal{K}$, is connected to all nodes $n \in \mathcal{B}\cup\mathcal{T}$ and collects all datapoints (both correctly and incorrectly classified) with predicted class k.

A formal definition for the flow graph associated with a decision tree that can be used to train imbalanced decision trees and that can track all datapoints is as follows.

Definition 6 (Complete Flow Graph of an Imbalanced Decision Tree). Given a decision tree of depth d, we define the directed *complete flow graph* $\mathcal{G} = (\mathcal{V}, \mathcal{A})$ associated with the tree that can be used to learn imbalanced decision trees of maximum depth d and that tracks all datapoints through the graph as follows. Let $\mathcal{V} := \{s\} \cup \{t_k : k \in \mathcal{K}\} \cup \mathcal{B} \cup \mathcal{T}$ be the vertices of the flow graph. Given $n \in \mathcal{B}$, let $\ell(n)$, $r(n)$, and $a(n)$ be as in Definition 5 and let

$$\mathcal{A} := \{(n, \ell(n)) : n \in \mathcal{B}\} \cup \{(n, r(n)) : n \in \mathcal{B}\} \cup \{(s, 1)\} \cup \{(n, t_k) : n \in \mathcal{B}\cup\mathcal{T}, k \in \mathcal{K}\}$$

be the arcs of the graph.

We are now ready to formulate the variant of Problem (7) that tracks all datapoints through the flow graph with sink nodes t_k, $k \in \mathcal{K}$. In a way that parallels Formulation (7), we introduce auxiliary variables $z^i_{n,t_k} \in \{0,1\}$ for each node $n \in \mathcal{B}\cup\mathcal{T}$ and each class $k \in \mathcal{K}$ to track the flow of datapoints (both correctly classified and missclassified) to the sink node t_k. Our formulation reads

$$\text{maximize} \quad \sum_{i\in\mathcal{I}} \sum_{n\in\mathcal{B}\cup\mathcal{T}} z^i_{n,t_{y^i}} \tag{11a}$$

$$\text{subject to} \quad \sum_{f\in\mathcal{F}} b_{nf} + p_n + \sum_{m\in\mathcal{P}(n)} p_m = 1 \qquad \forall n \in \mathcal{B}, \tag{11b}$$

$$p_n + \sum_{m\in\mathcal{P}(n)} p_m = 1 \qquad \forall n \in \mathcal{T}, \tag{11c}$$

$$z^i_{a(n),n} = z^i_{n,\ell(n)} + z^i_{n,r(n)} + \sum_{k\in\mathcal{K}} z^i_{n,t_k} \qquad \forall n \in \mathcal{B}, i \in \mathcal{I}, \tag{11d}$$

$$z^i_{a(n),n} = \sum_{k\in\mathcal{K}} z^i_{n,t_k} \qquad \forall n \in \mathcal{T}, i \in \mathcal{I}, \tag{11e}$$

$$z^i_{s,1} = 1 \qquad \forall i \in \mathcal{I}, \tag{11f}$$

$$z^i_{n,\ell(n)} \leq \sum_{f\in\mathcal{F}: x^i_f = 0} b_{nf} \qquad \forall n \in \mathcal{B}, i \in \mathcal{I}, \tag{11g}$$

$$z^i_{n,r(n)} \leq \sum_{f\in\mathcal{F}: x^i_f = 1} b_{nf} \qquad \forall n \in \mathcal{B}, i \in \mathcal{I}, \tag{11h}$$

$$z^i_{n,t_k} \leq w^n_k \qquad \forall n \in \mathcal{B}\cup\mathcal{T}, k \in \mathcal{K}, i \in \mathcal{I}, \tag{11i}$$

$$\sum_{k\in\mathcal{K}} w^n_k = p_n \qquad \forall n \in \mathcal{B}\cup\mathcal{T}, \tag{11j}$$

$$w^n_k \in \{0,1\} \qquad \forall n \in \mathcal{B}\cup\mathcal{T}, k \in \mathcal{K}, \tag{11k}$$

$$b_{nf} \in \{0,1\} \qquad \forall n \in \mathcal{B}, f \in \mathcal{F}, \tag{11l}$$

$$p_n \in \{0,1\} \qquad \forall n \in \mathcal{B}\cup\mathcal{T}, \tag{11m}$$

$$z^i_{a(n),n}, z^i_{n,t_k} \in \{0,1\} \qquad \forall n \in \mathcal{B}\cup\mathcal{T}, i \in \mathcal{I}, k \in \mathcal{K}. \tag{11n}$$

An interpretation of the problem constraints is as follows. Constraints (11b)–(11j) exactly mirror Constraints (7b)–(7j) in Problem (7). There are three main differences in these constraints. Flow conservation Constraints (11d) and (11e) now allow flow to be directed to any one of the sink nodes t_k from any one of the nodes $n \in \mathcal{B} \cup \mathcal{T}$. Constraints (11f) ensure that the flow incoming into the source and associated with each datapoint equals one. Constraints (11i) stipulate that a datapoint is only allowed to be directed to the sink corresponding to the class predicted at the leaf where the datapoint landed. Constraints (11d)–(11f) and (11i) together ensure that all datapoints get routed to the sink associated with their predicted class. As a result of these changes, sink node t_k collects *all* datapoints, whether correctly classified or not, with predicted class $k \in \mathcal{K}$. Objective (11a) is updated to reflect that a datapoint is correctly classified if and only if it flows to sink t_{y^i}.

At a feasible solution to Problem (11), the quantity $\sum_{i \in \mathcal{I}} z^i_{a(n),n}$ represents the total number of datapoints that land at node n. The number of datapoints of class $k \in \mathcal{K}$ that are correctly (respectively, incorrectly) classified is expressible as $\sum_{i \in \mathcal{I}: y^i = k} \sum_{n \in \mathcal{B} \cup \mathcal{T}} z^i_{n,t_k}$ (respectively, $|\{i \in \mathcal{I} : y^i = k\}| - \sum_{i \in \mathcal{I}: y^i = k} \sum_{n \in \mathcal{B} \cup \mathcal{T}} z^i_{n,t_k}$), whereas the total number of datapoints for which we predict class k can be written as $\sum_{i \in \mathcal{I}} \sum_{n \in \mathcal{B} \cup \mathcal{T}} z^i_{n,t_k}$.

Problem (11) allows us to learn meaningful decision trees for the case of imbalanced data by modifying the objective function of the problem and/or by augmenting this nominal formulation with constraints. We detail these variants in what follows.

4.2.1. Balanced Accuracy. A common approach to handle imbalanced data sets is to optimize the so-called *balanced accuracy*, which averages the accuracy across classes. Intuitively, in the case of two classes, it corresponds to the average of the true positive and true negative rates (Mower 2005). It can also be viewed as the average between sensitivity and specificity. Maximizing balanced accuracy can be achieved by replacing the objective function of Problem (11) with

$$\frac{1}{|\mathcal{K}|} \sum_{k \in \mathcal{K}} \frac{1}{|\{i \in \mathcal{I} : y^i = k\}|} \sum_{i \in \mathcal{I}: y^i = k} \sum_{n \in \mathcal{B} \cup \mathcal{T}} z^i_{n,t_k}.$$

Each term in the previous summation represents the proportion of datapoints from class k that are correctly classified. Having high balanced accuracy is important when dealing with imbalanced data and correctly predicting all classes is (equally) important.

4.2.2. Worst-Case Accuracy. As a variant to optimizing the average accuracy across classes, we propose to maximize the worst-case (minimum) accuracy. This can be achieved by replacing the objective function of Problem (11) with

$$\min_{k \in \mathcal{K}} \frac{1}{|\{i \in \mathcal{I} : y^i = k\}|} \sum_{i \in \mathcal{I}: y^i = k} \sum_{n \in \mathcal{B} \cup \mathcal{T}} z^i_{n,t_k}.$$

To the best of our knowledge, this idea has not been proposed in the literature. Compared with minimizing average accuracy, this objective attaches more importance to the class that is "worse-off." Having high worst-case accuracy is important when dealing with imbalanced data and correctly predicting the class that is the hardest to predict is important. This is the case for example when diagnosing cancer where positive cases are hard to identify and having large false-negative rates (i.e., low true positive rate) could cost a patient's life. There are also related works on robust classification where the objective happens to be the minimization of worst-case accuracy over noisy features/labels (Bertsimas et al. 2019a) or over adversarial examples (Vos and Verwer 2022).

In the remaining of this section we focus on the case of binary classification where $\mathcal{K} = \{0, 1\}$ and we refer to $k = 1$ (respectively, $k = 0$) as the positive (respectively, negative) class. In this setting we can discuss metrics such as recall, precision, sensitivity, and specificity more conveniently. Naturally, these definitions can be generalized to cases with more classes.

4.2.3. Constraining Recall. An important metric in the classification problem when dealing with imbalanced data sets is recall (also referred to as sensitivity), which is the fraction of all datapoints from the positive class that are correctly identified. Guaranteeing a certain level $C \in [0, 1]$ of recall can be achieved by augmenting Problem (11) with the constraint

$$\frac{1}{|\{i \in \mathcal{I} : y^i = 1\}|} \sum_{i \in \mathcal{I}: y^i = 1} \sum_{n \in \mathcal{B} \cup \mathcal{T}} z^i_{n,t_1} \geq C.$$

Decreasing the number of false negatives (datapoints from the positive class that are incorrectly predicted) increases recall. Thus, in cases where false negatives can have dramatic consequences and even cost lives, such as in cancer diagnosis or when predicting landslides, guaranteeing a certain level of recall can help mitigate such risks.

4.2.4. Constraining Precision. Our method can conveniently be used to learn decision trees that have sufficiently high precision, which is defined as the portion of correctly classified datapoints from the positive class out of all datapoints with positive predicted class. This can be achieved by augmenting Problem (11) with the constraint

$$\sum_{i \in \mathcal{I}: y^i = 1} \sum_{n \in \mathcal{B} \cup \mathcal{T}} z^i_{n,t_1} \geq C \sum_{i \in \mathcal{I}} \sum_{n \in \mathcal{B} \cup \mathcal{T}} z^i_{n,t_1}, \tag{12}$$

where $C \in [0,1]$ is a hyperparameter that represents the minimum acceptable precision that can be tuned via cross-validation. Decreasing the number of false positives (datapoints from the negative class that are incorrectly classified) increases precision. Thus, constraining precision is useful in settings where having a low number of false negatives is not as important and having a low number of false positives, such as when making product recommendations.

4.2.5. Balancing Sensitivity and Specificity. Günlük et al. (2021) address the issue of imbalanced data by maximizing sensitivity (true-positive rate) in the objective while guaranteeing a certain level of specificity (true-negative rate), or the converse, instead of optimizing the total accuracy. In the case of binary classification where $\mathcal{K} = \{0,1\}$, we can constrain specificity from below by augmenting Problem (11) with the constraint

$$\frac{1}{|\{i \in \mathcal{I} : y^i = 0\}|} \sum_{i \in \mathcal{I}: y^i = 0} \sum_{n \in \mathcal{B} \cup \mathcal{T}} z^i_{n,t_0} \geq C,$$

and maximize sensitivity by replacing its objective function with

$$\frac{1}{|\{i \in \mathcal{I} : y^i = 1\}|} \sum_{i \in \mathcal{I}: y^i = 1} \sum_{n \in \mathcal{B} \cup \mathcal{T}} z^i_{n,t_1}.$$

This method is useful in settings such as infectious disease testing wherein we want to maximize the chance of correctly predicting someone to be infectious while making sure to identify noninfectious individuals with a high enough confidence.

4.3. Solution Approach for Generalizations

We now briefly discuss how to solve Formulation (7) (for learning imbalanced decision trees) and Formulation (11) (which tracks all datapoints through the flow graph) and their variants introduced in Sections 4.1 and 4.2 and Online Appendix EC.1.

Nominal Formulations (7) and (11) and all their variants augmented with constraints that *do not* couple datapoints with one another (e.g., sparsity Constraints (8) or interpretability Constraints (9)) can be effectively solved using Benders' decomposition. This can be achieved by adapting Algorithm 1 from Section 3. Conversely, when these formulations are augmented with constraints that *do* couple datapoints with one another (like the fairness constraints from Online Appendix EC.1), Benders' decomposition is no longer applicable. In such cases, these formulations need to be solved directly as monolithic MIO problems.

The Benders' decomposition approach for solving Formulation (7) and the corresponding variants is provided in Online Appendix EC.6. The derivation of the Benders' decomposition for Formulation (11) and the corresponding variants is left for the reader.

5. Experiments

In the following section, we discuss the data sets we use in our experiments, the approaches that we compare with, the experimental setup, and our findings. More extensive numerical results are included in the Online Appendix.

5.1. Benchmark Approaches and Data Sets

In our numerical experiments, we have two sets of experiments: one on data sets with only categorical features and one on data sets with a mixture of categorical and real-valued features. We now describe both sets of data and the approaches we compare with in each case.

5.1.1. Data Sets with Only Categorical Features. In the first part of our experiments, we use all 12 publicly available data sets with only categorical features from the UCI data repository (Dua and Graff 2017) as detailed in Table 1. We compare the flow-based formulation (`FlowOCT`) given in Problem (7) and its Benders' decomposition (`BendersOCT`) described in Online Appendix EC.6 to the univariate splits formulations proposed by Bertsimas and Dunn (2017) (`OCT`) and Verwer and Zhang (2019) (`BinOCT`). We also compare with the heuristic algorithm based on local search for solving `OCT` proposed by Bertsimas and Dunn (2019) (`LST`). As the code used for `OCT` is not publicly available, we implemented the corresponding formulation (adapted to the case of binary data). The details of this implementation are given in Online Appendix EC.3. We also used the `Python` implementation of `LST` that is available for academic use. In `LST`, we set the complexity parameter `cp` to zero that is analogous to setting $\lambda = 0$. To make the optimality gap comparable across approaches, we made some adjustments to

Table 1. Benchmark Data Sets with Only Categorical Features, Along with Their Number of Rows ($|\mathcal{I}|$), Number of Features ($|\mathcal{F}|$), and Number of Classes ($|\mathcal{K}|$)

Data set	$\lvert\mathcal{I}\rvert$	$\lvert\mathcal{F}\rvert$	$\lvert\mathcal{K}\rvert$
soybean-small	47	45	4
monk3	122	15	2
monk1	124	15	2
hayes-roth	132	15	3
monk2	169	15	2
house-votes-84	232	16	2
spect	267	22	2
breast-cancer	277	38	2
balance-scale	625	20	3
tic-tac-toe	958	27	2
car-evaluation	1,728	20	4
kr-vs-kp	3,196	38	2

Table 2. Benchmark Data Sets with Mixed Features, Along with Their Number of Rows ($|\mathcal{I}|$), Number of Features ($|\mathcal{F}|$), and Number of Classes ($|\mathcal{K}|$)

Data set	$\lvert\mathcal{I}\rvert$	$\lvert\mathcal{F}\rvert$ for OCT	$\lvert\mathcal{F}\rvert$ for BendersOCT-5	$\lvert\mathcal{F}\rvert$ for BendersOCT-10	$\lvert\mathcal{K}\rvert$
echocardiogram	61	8	32	61	2
hepatitis	80	19	43	68	2
fertility	100	20	28	28	2
iris	150	4	20	38	3
wine	178	13	65	130	3
planning-relax	182	12	60	120	2
breast-cancer-prognostic	194	33	164	321	2
parkinsons	195	22	110	218	2
connectionist-bench-sonar	208	60	300	600	2
seeds	210	7	35	70	3
cylinder-bands	277	257	314	370	2
heart-cleveland	297	22	44	68	5
ionosphere	351	33	157	298	2
thoracic-surgery	470	27	39	54	2
climate	540	18	90	180	2
breast-cancer-diagnostic	569	30	150	300	2
indian-liver-patient	579	10	45	88	2
credit-approval	653	42	64	89	2
blood-transfusion	748	4	20	36	2
diabetes	768	8	39	75	2
qsar-biodegradation	1,055	41	139	234	2
banknote-authentication	1,372	4	20	40	2
ozone-level-detection-one	1,848	72	358	714	2
image-segmentation	2,310	18	82	164	7
seismic-bumps	2,584	19	48	73	2
thyroid-disease-ann-thyroid	3,772	21	45	74	3
spambase	4,601	57	108	190	2
wall-following-robot-2	5,456	24	116	228	4

the objective function of FlowOCT and BendersOCT by subtracting number of total datapoints from the objective, such that, for all approaches, the objective value reflects the number of misclassified datapoints. The results for worst-case accuracy objective discussed in Section 4.2 can be found in Online Appendix EC.8.2. We also numerically analyze the strength of all formulations by looking at their LO relaxation (see Online Appendix EC.8.3). Analysis of some implementation variants of BendersOCT (see Remark 2 and Online Appendix EC.8.1).

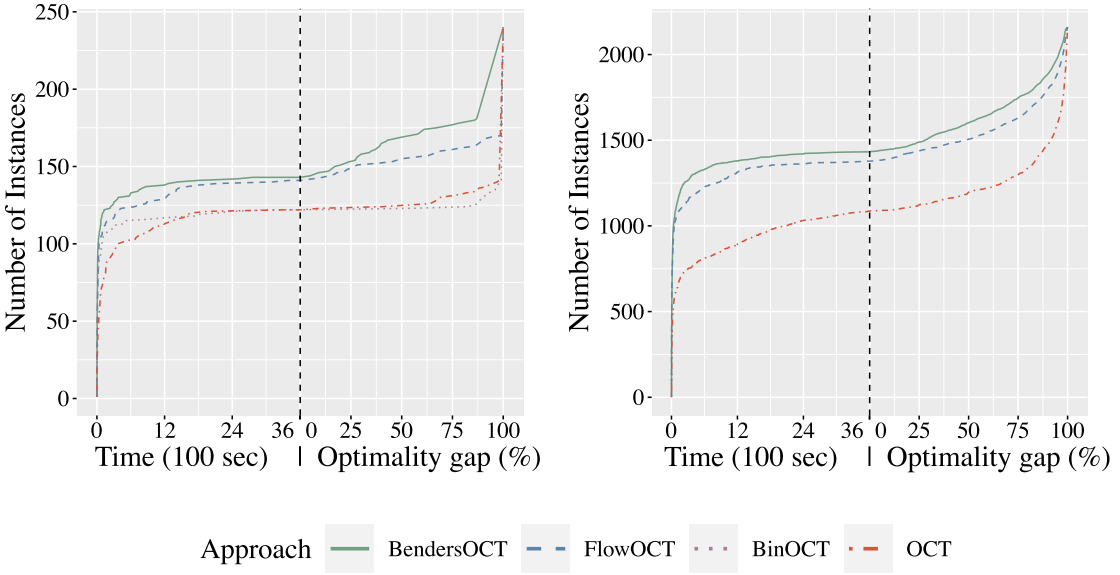
5.1.2. Data Sets with Mixed Features. In the second part of our experiments, we use 28 publicly available data sets with both categorical and real-valued features from the UCI data repository as detailed in Table 2. In this part, we compare BendersOCT to OCT. We did not include BinOCT as it cannot handle real-valued features. We also did not include FlowOCT as it is outperformed by BendersOCT. To run BendersOCT on these data sets, we first discretize the real-valued features into 5 and 10 buckets (quantiles) and then one-hot encode the discretized columns. We refer to the version of BendersOCT that we run on the discretized data with 5 (respectively, 10) buckets as BendersOCT-5 (respectively, BendersOCT-10). As OCT can handle real-valued features without special encoding, we use the original format of the data sets for OCT. For both OCT and BendersOCT, the categorical features are one-hot encoded.

5.2. Experimental Setup

For each data set, we create five random splits of the data each consisting of a training set (50%), a calibration set (25%) used to calibrate the hyperparameters, and a test set (25%). For each split, for each depth $d \in \{2,3,4,5\}$, and for each choice of regularization parameter $\lambda \in \{0, 0.1, 0.2, \ldots 0.9\}$, we train a decision tree. For any given depth we calibrate λ on the calibration set. Having the best λ for any given split and depth, we train a decision tree on the union of the training and calibration sets and report the out-of-sample performance on the test set.

All approaches are implemented in Python programming language and solved using Gurobi 8.1 (Gurobi 2015). All problems are solved on a single core of SL250s Xeon CPUs and 4 GB of memory with a 60-minute time limit. Our implementation of FlowOCT and BendersOCT can be found online at https://github.com/D3M-Research-Group/StrongTree along

Figure 4. (Color online) Left (Respectively, Right) Figure Shows for Balanced (Respectively, Imbalanced) Decision Trees the Number of Instances Solved to Optimality by Each Approach Within a Given Time on the Time Axis and the Number of Instances with Optimality Gap No Larger Than Each Given Value at the Time Limit on the Optimality Gap Axis



with instructions and is freely distributed for academic and nonprofit use. In what follows, we discuss the computational efficiency and statistical performance of the different MIO approaches.

5.3. Results on Categorical Data Sets

5.3.1. In-Sample (Optimization) Performance. Figure 4 summarizes the in-sample performance of all methods. Detailed results are provided in Online Appendix EC.7. From the time axis of Figure 4 (left), we observe that for the case of balanced decision trees, `BinOCT` and `OCT` are able to solve 122 instances (out of 240) within the time limit, but `BendersOCT` solves the same number of instances in only 125 seconds, resulting in a $\lfloor \frac{3,600}{125} \rfloor = 29\times$ speedup. Similarly, from the time axis of Figure 4 (right), it can be seen that in the case of imbalanced decision trees, `OCT` is able to solve 1087 instances (out of 2400) within the time limit, whereas `BendersOCT` requires only 71 seconds to do so, resulting in a $51\times$ speedup. `BinOCT`'s implementation does not allow for imbalanced decision trees, so it is excluded from Figure 4 (right). From the optimality gap axis of Figure 4 (left), we observe that `BendersOCT` and `FlowOCT` both achieve better optimality gaps than either of `BinOCT` or `OCT`. Similarly, from the optimality gap axis of Figure 4 (right), we observe the smaller optimality gap of `BendersOCT` and `FlowOCT` compared with `OCT`, when we optimize over imbalanced decision trees.

Our experiments also demonstrate the limitations of our approaches. We observe that `BendersOCT` successfully solves almost all the MIO instances up to the data set "spect" (consisting of 267 datapoints and 22 features) within the one-hour time limit, regardless of the chosen depth. However, when dealing with larger data sets like "kr-vs-kp" (consisting of 3,196 datapoints and 38 features), we are unable to find the optimal solution for depths greater than three. For instance, when considering a depth of five for the "kr-vs-kp" data set, we observe an average optimality gap of 93%. It is worth noting that our approach achieves an out-of-sample accuracy of 89% in this instance, surpassing the performance of both `BinOCT` (87%) and `OCT` (66%).

5.3.2. Out-of-Sample Performance. Table 3 summarizes the out-of-sample performance of all methods. Detailed results are reported in Online Appendix EC.7. From the table we observe that the better optimization performance translates to superior out-of-sample properties as well: of 48 instances (average accuracy across five samples for each data set and depth given the calibrated λ), `OCT` achieves the best out-of-sample accuracy in seven instances (excluding ties), `BinOCT` in eight, and the new formulation `BendersOCT` (respectively, `FlowOCT`) achieves the best accuracy in nine (respectively, eight) instances. `BendersOCT` (respectively, `FlowOCT`) improves out-of-sample accuracy with respect to `BinOCT` and `OCT` by up to 8% (respectively, 7%) and 36% (respectively, 21%), respectively.

5.3.3. Comparing with `LST`. We compare `BendersOCT` (with $\lambda = 0$) to the state-of-the-art approach `LST`, which is based on local search, on 240 MIO instances (which consist of all 12 data sets, 5 splits per data set, and 4 different depths). On the one hand, `LST` is much

Table 3. Summary of the Out-of-Sample Performance of All Methods on Categorical Data Sets

Approach	Best accuracy (of 48, excluding ties)	Average accuracy	Maximum accuracy improvement
OCT	7	0.76 ± 0.14	—
BinOCT	8	0.79 ± 0.13	—
FlowOCT	8	0.79 ± 0.13	7% (21%) w.r.p. to BinOCT (OCT)
BendersOCT	9	0.80 ± 0.14	8% (36%) w.r.p. to BinOCT (OCT)

faster, requiring only seconds to find a local optimum (does not guarantee optimality). The average solving time among all 240 instances for LST (respectively, BendersOCT) is 0.52 (respectively, 1,539) seconds. On the other hand, BendersOCT can solve 143 of the instances to provable optimality: Of those, LST is able to find an optimal solution (without certificate) in 117 and produces a suboptimal (by 1% in average in-sample accuracy) solution in the remaining 26. The average solving time among the 117 instances that both approaches solve to optimality, for LST (respectively, BendersOCT) is 0.26 (respectively, 80) seconds. Overall, of the 240 instances (including those not solved to optimality), BendersOCT produces a better solution in 67 instances (by 1% in average in-sample accuracy), whereas LST outperforms BendersOCT in 37 instances (by 2% in average in-sample accuracy), with the remaining 136 being tied between the two methods. Thus, we conclude that LST is a method able to deliver high-quality solutions very fast; however, if enough computational resources are available, the exact BendersOCT is able to deliver better solutions overall. Table 4 reports a summary of these results. Detailed results are provided in Online Appendix EC.7.

5.4. Results on Mixed-Feature Data Sets

5.4.1. In-Sample Performance. First, we point that OCT results in numerical issues performance in over half of the instances tested. In our experiments, we found out that, although the formulations described Bertsimas and Dunn (2017) are correct *if solved on real arithmetic*, they may fail to produce optimal solutions with solvers working with numerical tolerances. In particular, in our experiments with Gurobi, we found out that in many cases the solver terminates with a solution it claims as optimal (often in seconds, with a solution that allegedly correctly classifies all points), but upon further inspection, the tree described by the decision variables $(\boldsymbol{b}, \boldsymbol{w})$ misclassifies most of the points. A detailed discussion on this matter, along with an example, can be found in Online Appendix EC.4. Thus, we do not report computational times of OCT (because several instances that are solved very fast are in fact considerably suboptimal), but we do report out-of-sample performance corresponding to the solution found by the solver.

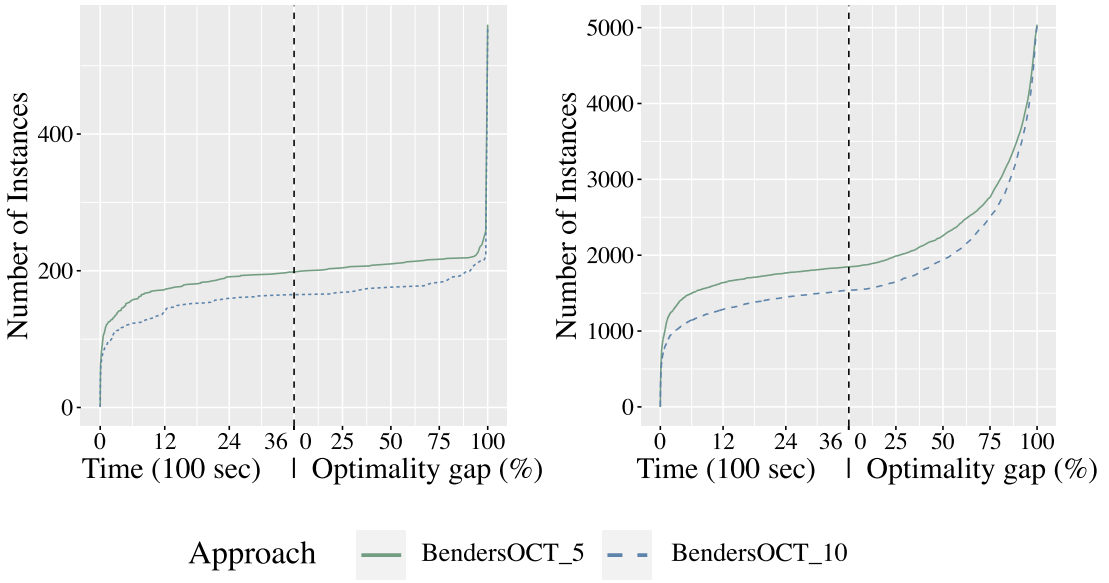
Figure 5 summarizes the in-sample performance of BendersOCT-5 and BendersOCT-10. Detailed results can be found in Online Appendix EC.7. From the figure, we observe that adding more buckets to the discretization process increases the computational time. This outcome is expected because adding more features leads to a linear growth in the size of the MIO formulation. When comparing average in-sample accuracy, using 10 buckets does not offer an advantage over 5 buckets; in fact, it slightly underperforms. For balanced (respectively, imbalanced) decision trees, using 5 buckets improves in-sample accuracy from 0.8872 to 0.8914 (respectively, from 0.8594 to 0.8614) while reducing computational time by 7% (respectively, 9%). Using 10 buckets in large instances may hamper solvers, and thus using just 5 buckets results in better feasible solutions found within the time limit.

5.4.2. Out-of-Sample Performance. Table 5 summarizes the out-of-sample results on the mixed-features data sets. Detailed results are reported in Online Appendix EC.7. We see that in terms of out-of-sample accuracy,

Table 4. Summary of the Comparison of the In-Sample Results of LST vs. BendersOCT on Categorical Data Sets

Metric	LST	BendersOCT
Average solving time	0.52 s	1,539 s
Average in-sample accuracy	89%	89%
Instances solved to optimality	117 (without certificate)	143
Average solving time (117 optimal instances)	0.26 s	80 s
Better solutions (excluding ties)	37	67

Figure 5. (Color online) Left (Respectively, Right) Figure Shows for Balanced (Respectively, Imbalanced) Decision Trees the Number of Instances Solved to Optimality by Each Approach on the Time Axis and the Number of Instances with Optimality Gap No Larger Than Each Given Value at the Time Limit on the Optimality Gap Axis



Notes. OCT is not included in this figure because of its numerical instabilities due to having "little-m" constraints that caused discrepancy between the optimal objective value and the actual in-sample accuracy. Refer to Online Appendix EC.4 for further information.

out of 112 instances (average accuracy across five samples for each data set and depth given the calibrated λ), OCT is the best method in 25 instances (excluding ties), whereas BendersOCT-5 and BendersOCT-10 are better in 71. Furthermore, BendersOCT-5 (BendersOCT-10) improves out-of-sample accuracy with respect to OCT by up to 50% (51%). Therefore, despite the fact that we are losing some information by discretizing the features, BendersOCT can still output higher quality solutions compared with OCT. Moreover, the gains achieved by BendersOCT-10 over BendersOCT-5 are on average small, suggesting that a coarse discretization is sufficient in practice. One could justify these findings by interpreting the discretization as some form of regularization that helps avoiding overfitting and yields better out-of-sample performance.

6. Conclusion

We proposed a new MIO formulation for classification trees with univariate splits with a stronger LO relaxation than state-of-the-art MIO-based approaches. We also provided a tailored Benders' decomposition method to speed-up the computations. Our experiments reveal better computational performance than state-of-the-art methods including 29 (respectively, 51) times speedup when we optimize over balanced (respectively, imbalanced) trees. These also translated to improved out-of-sample performance up to 36%. We showcase the modeling power of our framework to deal with imbalanced data sets and to design interpretable and fair decision trees. Our model can also act as a building block for more sophisticated predictive and prescriptive tasks. For example, variants of our method can be used to learn optimal prescriptive trees from observational data (Jo et al. 2021) or optimal robust classification trees (Justin et al. 2022).

Acknowledgments

The authors thank the anonymous reviewers for valuable and constructive feedback that improved the quality of this paper.

Table 5. Summary of the Out-of-Sample Performance of Various Approaches on Mixed-Feature Data Sets Given the Calibrated λ

Approach	Best accuracy (of 112, excluding ties)	Average accuracy	Maximum accuracy improvement
OCT	25	0.75 ± 0.18	—
BendersOCT-5	37	0.81 ± 0.12	50% w.r.p. to OCT
BendersOCT-10	34	0.81 ± 0.13	51% w.r.p. to OCT

References

Aghaei S, Azizi MJ, Vayanos P (2019) Learning optimal and fair decision trees for non-discriminative decision-making. *Proc. Conf. AAAI Artificial Intelligence* 33:1418–1426.

Aglin G, Nijssen S, Schaus P (2020) Learning optimal decision trees using caching branch-and-bound search. *Proc. AAAI Conf. Artificial Intelligence* 34(4):3146–3153.

Anderson R, Huchette J, Ma W, Tjandraatmadja C, Vielma JP (2020) Strong mixed-integer programming formulations for trained neural networks. *Math. Programming* 183(1–2):3–39.

Atamturk A, Gomez A, Han S (2021) Sparse and smooth signal estimation: Convexification of l0-formulations. *J. Machine Learn. Res.* 22(52):1–43.

Azizi MJ, Vayanos P, Wilder B, Rice E, Tambe M (2018) Designing fair, efficient, and interpretable policies for prioritizing homeless youth for housing resources. *Proc. Internat. Conf. Integration Constraint Programming Artificial Intelligence Oper. Res.* (Springer, Berlin), 35–51.

Benders JF (1962) Partitioning procedures for solving mixed-variables programming problems. *Numerical Math.* 4(1):238–252.

Bertsimas D, Dunn J (2017) Optimal classification trees. *Machine Learn.* 106(7):1039–1082.

Bertsimas D, Dunn J (2019) *Machine Learning Under a Modern Optimization Lens* (Dynamic Ideas, Charlestown, MA).

Bertsimas D, Stellato B (2021) The voice of optimization. *Machine Learn.* 110(2):249–277.

Bertsimas D, Van Parys B (2020) Sparse high-dimensional regression: Exact scalable algorithms and phase transitions. *Ann. Statist.* 48(1):300–323.

Bertsimas D, Dunn J, Pawlowski C, Zhuo YD (2019a) Robust classification. *INFORMS J. Optim.* 1(1):2–34.

Bertsimas D, Kung J, Trichakis N, Wang Y, Hirose R, Vagefi PA (2019b) Development and validation of an optimized prediction of mortality for candidates awaiting liver transplantation. *Amer. J. Transplantation* 19(4):1109–1118.

Bienstock D, Muñoz G, Pokutta S (2018) Principled deep neural network training through linear programming. Preprint, submitted October 7, https://arxiv.org/abs/1810.03218.

Biggs M, Hariss R (2018) Optimizing objective functions determined from random forests. Preprint, submitted June 16, https://dx.doi.org/10.2139/ssrn.2986630.

Bishop CM (2006) *Pattern Recognition and Machine Learning (Information Science and Statistics)* (Springer-Verlag, Berlin).

Bixby RE (2012) A brief history of linear and mixed-integer programming computation. *Document Math.* Extra Volume ISMP (2012):107–121.

Blanquero R, Carrizosa E, Molero-Río C, Romero Morales D (2020) Sparsity in optimal randomized classification trees. *Eur. J. Oper. Res.* 284(1):255–272.

Blanquero R, Carrizosa E, Molero-Río C, Romero Morales D (2021) Optimal randomized classification trees. *Comput. Oper. Res.* 132:105281.

Blurock ES (1995) Automatic learning of chemical concepts: Research octane number and molecular substructures. *Comput. Chemistry* 19(2):91–99.

Breiman L (1996) Bagging predictors. *Machine Learn.* 24(2):123–140.

Breiman L (2001) Random forests. *Machine Learn.* 45(1):5–32.

Breiman L, Friedman JH, Olshen RA, Stone CJ (1984) *Classification and Regression Trees* (Wadsworth and Brooks, Monterey, CA).

Carrizosa E, Molero-Río C, Romero Morales D (2021) Mathematical optimization in classification and regression trees. *TOP* 29(1):5–33.

Chan H, Rice E, Vayanos P, Tambe M, Morton M (2018) From empirical analysis to public policy: Evaluating housing systems for homeless youth. *Proc. Joint Eur. Conf. Machine Learn. Knowledge Discovery Databases* (Springer, Berlin), 69–85.

Chen X, Zhu CC, Yin J (2019) Ensemble of decision tree reveals potential mirna-disease associations. *PLOS Comput. Biology* 15(7):e1007209.

Ciocan DF, Mišić VV (2022) Interpretable optimal stopping. *Management Sci.* 68(3):1616–1638.

CPLEX II (2009) V12. 1: User's manual for CPLEX. *Internat. Bus. Machines Corporation* 46(53):157.

Demirović E, Stuckey PJ (2021) Optimal decision trees for nonlinear metrics. *Proc. Conf. AAAI Artificial Intelligence* 35:3733–3741.

Demirović E, Lukina A, Hebrard E, Chan J, Bailey J, Leckie C, Ramamohanarao K, et al. (2020) Murtree: Optimal classification trees via dynamic programming and search. Preprint, submitted July 24, https://arxiv.org/abs/2007.12652.

Detassis F, Lombardi M, Milano M (2020) Teaching the old dog new tricks: Supervised learning with constraints. Preprint, submitted February 25, https://arxiv.org/abs/2002.10766.

Dong H, Chen K, Linderoth J (2015) Regularization vs. relaxation: A conic optimization perspective of statistical variable selection. Preprint, submitted October 20, https://arxiv.org/abs/1510.06083.

Dua D, Graff C (2017) UCI machine learning repository. Accessed September 1, 2019, http://archive.ics.uci.edu/ml.

Gade D, Küçükyavuz S, Sen S (2014) Decomposition algorithms with parametric gomory cuts for two-stage stochastic integer programs. *Math. Programming* 144(1–2):39–64.

Gangammanavar H, Liu Y, Sen S (2021) Stochastic decomposition for two-stage stochastic linear programs with random cost coefficients. *INFORMS J. Comput.* 33(1):51–71.

Goldberg AV, Tarjan RE (1988) A new approach to the maximum-flow problem. *J. ACM* 35(4):921–940.

Gómez A (2021) Outlier detection in time series via mixed-integer conic quadratic optimization. *SIAM J. Optim.* 31(3):1897–1925.

Günlük O, Kalagnanam J, Li M, Menickelly M, Scheinberg K (2021) Optimal decision trees for categorical data via integer programming. *J. Global Optim.* 81(1):233–260.

Guo C, Bodur M, Aleman DM, Urbach DR (2021) Logic-based benders decomposition and binary decision diagram based approaches for stochastic distributed operating room scheduling. *INFORMS J. Comput.* 33(4):1551–1569.

Gurobi I (2015) Gurobi optimizer reference manual. Accessed September 1, 2019, http://www.gurobi.com.

Hazimeh H, Mazumder R, Saab A (2022) Sparse regression at scale: Branch-and-bound rooted in first-order optimization. *Math. Programming* 196(1–2):347–388.

Hochbaum DS (2008) The pseudoflow algorithm: A new algorithm for the maximum-flow problem. *Oper. Res.* 56(4):992–1009.

Hu TC (1963) Multi-commodity network flows. *Oper. Res.* 11(3): 344–360.

Hu X, Rudin C, Seltzer M (2019) Optimal sparse decision trees. *Adv. Neural Inform. Processing Systems* 32:7267–7275.

Hyafil L, Rivest RL (1976) Constructing optimal binary search trees is NP complete. *Inform. Processing Lett.* 5(1):15–17.

Intrator J, Allan E, Palmer M (1992) Decision tree for the management of substance-abusing psychiatric patients. *J. Substance Abuse Treatment* 9(3):215–220.

Jo N, Aghaei S, Gómez A, Vayanos P (2021) Learning optimal prescriptive trees from observational data. Preprint, submitted August 31, https://arxiv.org/abs/2108.13628.

Justin N, Aghaei S, Gómez A, Vayanos P (2022) Optimal robust classification trees. *Proc. 36th AAAI Conf. Artificial Intelligence Workshop Adversarial Machine Learn. Beyond* (Curran Associates Inc., Red Hook, NY).

Khalilia M, Chakraborty S, Popescu M (2011) Predicting disease risks from highly imbalanced data using random forest. *BMC Medical Inform. Decision Making* 11(1):51.

Kirschbaum D, Adler R, Hong Y, Lerner-Lam A (2009) Evaluation of a preliminary satellite-based landslide hazard algorithm using

global landslide inventories. *Natural Hazards Earth Systems Sci.* 9(3):673–686.

Kuhn M, Weston S, Culp M, Coulter N, Quinlan R (2023a) C50: C5.0 Decision trees and rule-based models. Accessed July 25, 2024, https://CRAN.R-project.org/package=C50.

Kuhn M, Weston S, Culp M, Coulter N, Quinlan R (2023b) rpart: Recursive partitioning and regression trees, R package version 4.1.23. Accessed July 25, 2024, https://CRAN.R-project.org/package=rpart.

Liaw A, Wiener M (2002) Classification and regression by randomforest. *R News* 2(3):18–22.

Lin J, Zhong C, Hu D, Rudin C, Seltzer M (2020) Generalized and scalable optimal sparse decision trees. *Proc. Internat. Conf. Machine Learn.* (PMLR, New York), 6150–6160.

Liu J, Sen S (2020) Asymptotic results of stochastic decomposition for two-stage stochastic quadratic programming. *SIAM J. Optim.* 30(1):823–852.

Liu X, Küçükyavuz S, Luedtke J (2016) Decomposition algorithms for two-stage chance-constrained programs. *Math. Programming* 157(1):219–243.

Lombardi M, Baldo F, Borghesi A, Milano M (2020) An analysis of regularized approaches for constrained machine learning. Preprint, submitted May 20, https://arxiv.org/abs/2005.10674.

Lozano L, Smith JC (2022) A binary decision diagram based algorithm for solving a class of binary two-stage stochastic programs. *Math. Programming* 191(1):381–404.

MacNeil M, Bodur M (2022) Integer programming, constraint programming, and hybrid decomposition approaches to discretizable distance geometry problems. *INFORMS J. Comput.* 34(1): 297–314.

Magnanti TL, Wong RT (1981) Accelerating benders decomposition: Algorithmic enhancement and model selection criteria. *Oper. Res.* 29(3):464–484.

Mišić VV (2020) Optimization of tree ensembles. *Oper. Res.* 68(5): 1605–1624.

Mower JP (2005) Prep-Mt: Predictive RNA editor for plant mitochondrial genes. *BMC Bioinformatics* 6(1):96.

Narodytska N, Ignatiev A, Pereira F, Marques-Silva J (2018) Learning optimal decision trees with SAT. *Proc. Internat. Joint Conf. Artificial Intelligence* (AAAI Press, Palo Alto, CA), 1362–1368.

Nijssen S, Fromont E (2010) Optimal constraint-based decision tree induction from itemset lattices. *Data Mining Knowledge Discovery* 21(1):9–51.

Okada S, Ohzeki M, Taguchi S (2019) Efficient partition of integer optimization problems with one-hot encoding. *Sci. Rep.* 9(1): 1–12.

Olanow CW, Watts RL, Koller WC (2001) An algorithm (decision tree) for the management of Parkinson's disease (2001): Treatment guidelines. *Neurology* 56(suppl 5):S1–S88.

Quinlan JR (1986) Induction of decision trees. *Machine Learn.* 1(1): 81–106.

Quinlan JR (2014) *C4. 5: Programs for Machine Learning* (Elsevier, New York).

Rudin C (2019) Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Natural Machine Intelligence* 1(5):206–215.

Shaikhina T, Lowe D, Daga S, Briggs D, Higgins R, Khovanova N (2019) Decision tree and random forest models for outcome prediction in antibody incompatible kidney transplantation. *Biomedical Signal Processing Control* 52:456–462.

Vazirani VV (2013) *Approximation Algorithms* (Springer Science & Business Media, Boston).

Verhaeghe H, Nijssen S, Pesant G, Quimper CG, Schaus P (2020) Learning optimal decision trees using constraint programming. *Constraints* 25:226–250.

Verwer S, Zhang Y (2019) Learning optimal classification trees using a binary linear program formulation. *Proc. Conf. AAAI Artificial Intelligence* 33:1625–1632.

Vos D, Verwer S (2022) Robust optimal classification trees against adversarial examples. *Proc. AAAI Conf. Artificial Intelligence* 36(8): 8520–8528.

Wei W, Li J, Cao L, Ou Y, Chen J (2013) Effective detection of sophisticated online banking fraud on extremely imbalanced data. *World Wide Web (Bussum)* 16(4):449–475.

Xie W, Deng X (2020) Scalable algorithms for the sparse ridge regression. *SIAM J. Optim.* 30(4):3359–3386.

Sina Aghaei is a postdoctoral fellow at Harvard Kennedy School, specializing in the intersection of machine learning, optimization, and causal inference. His work focuses on creating robust, interpretable, and fair models for high-stakes domains.

Andrés Gómez is an assistant professor in the Department of Industrial and Systems Engineering at the University of Southern California. His research focuses on developing new theory and tools for challenging optimization problems arising in finance, machine learning, and statistics.

Phebe Vayanos is an associate professor of industrial & systems engineering and computer science at the University of Southern California (USC), where she holds a Viterbi early career chair in engineering. She is also co-director of the Center for Artificial Intelligence in Society at USC. Her research is focused on optimization and its interface with machine learning, causal inference, and economics. She is a recipient of the National Science Foundation CAREER award.